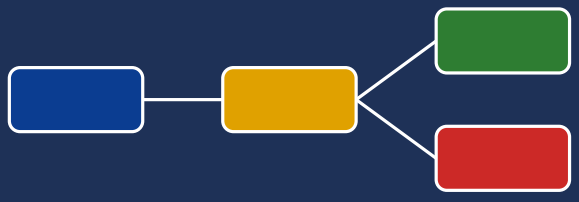
EXECUTION PLANS | PHYSICAL OPERATORS | JOINS | WAIT TYPES

SQL Server Execution Plans and Wait Types

A DBA's Reference and Playbook

How to read plans, every physical operator and join in depth, and what each wait type means, how to confirm it and how to fix it



WHAT YOU WILL LEARN

- How SQL Server compiles and runs a query; capturing and reading plans
- Cardinality estimation, statistics, warnings and red flags
- Complete physical operator reference and physical joins in depth
- Parallelism, memory grants and spills, common patterns, IQP
- Wait type encyclopedia, benign waits and troubleshooting playbooks

Prepared October 2026 for SQL Server 2016 to 2025. Features differ by version and build; verify details in Microsoft documentation.

Contents

How to Use This Guide . . . 6

1. How SQL Server Runs a Query . . . 7

1.1 From T-SQL text to results . . . 7

1.2 The optimizer is cost-based, not time-based . . . 7

1.3 How a plan runs: the iterator (pull) model . . . 8

1.4 Row mode and batch mode . . . 9

2. Capturing Execution Plans . . . 10

2.1 Types of plans . . . 10

2.2 Capturing in SSMS and with SET options . . . 10

2.3 Getting plans from the cache and Query Store . . . 11

2.4 Other sources and cautions . . . 11

3. How to Read an Execution Plan . . . 13

3.1 The ten-step method . . . 13

3.2 An annotated example . . . 13

3.3 Key operator properties . . . 14

3.4 Statement-level information on the root operator . . . 15

3.5 Seeks, scans and 'Number of Rows Read' . . . 15

4. Cardinality Estimation and Statistics . . . 17

4.1 Where estimates come from . . . 17

4.2 How estimates are built . . . 18

4.3 When statistics update . . . 18

4.4 Why estimates go wrong and what to do . . . 19

5. Plan Warnings and Red Flags . . . 20

5.1 Warnings shown in the plan . . . 20

5.2 Red flags without an icon . . . 21

5.3 Missing indexes: use with care . . . 21

6. Physical Operators: Complete Reference . . . 22

6.1 How to use this reference . . . 22

6.2 Reading data: scans, seeks and lookups . . . 23

6.3 Columnstore and batch mode . . . 24

6.4 Join operators (summary) . . . 24

6.5 Aggregation, sorting, ranking and windowing . . . 25

6.6 Row shaping: expressions, filters, asserts and unions . . . 26

6.7 Spools: temporary storage inside a plan . . . 26

6.8 Parallelism operators . . . 27
6.9 Data modification operators . . . 28
6.10 Remote, function and special-purpose operators . . . 29
6.11 Cursor operators . . . 29
6.12 Language elements and legacy operators . . . 30
6.13 Logical operators and what implements them . . . 31
7. Physical Joins in Depth . . . 32
7.1 The big picture . . . 32
7.2 Nested Loops Join . . . 33
7.3 Merge Join . . . 34
7.4 Hash Match (Hash Join) . . . 34
7.5 Adaptive Join . . . 35
7.6 Logical join types and physical support . . . 36
7.7 Semi joins and anti semi joins . . . 36
7.8 Join order, hints and when to force . . . 37
7.9 Join troubleshooting . . . 37
8. Parallelism in Plans . . . 38
8.1 Exchange operators . . . 38
8.2 When the optimizer chooses a parallel plan . . . 38
8.3 Reading a parallel plan . . . 39
8.4 Parallelism problems and tuning . . . 39
9. Memory Grants and Spills . . . 40
9.1 How a memory grant works . . . 40
9.2 Operators that use grant memory . . . 40
9.3 Spills . . . 41
9.4 Memory grant feedback and related IQP features . . . 41
9.5 Finding and fixing grant problems . . . 41
10. Common Plan Patterns and How to Fix Them . . . 43
10.1 Key lookups on large row counts . . . 43
10.2 Scan instead of seek: non-SARGable predicates . . . 43
10.3 Implicit conversions . . . 43
10.4 Parameter sniffing . . . 44
10.5 Local variables and the 'kitchen sink' search . . . 44
10.6 Table variables and multi-statement functions . . . 44
10.7 Correlated subqueries, spools and row-by-row work . . . 45
10.8 Sort spills and ORDER BY problems . . . 45
10.9 DISTINCT and GROUP BY that hide duplicates . . . 45

10.10 Paging with large offsets . . . 45
10.11 Window functions with the default frame . . . 45
10.12 Long IN / OR lists . . . 45
10.13 Big UPDATE and DELETE statements . . . 45
10.14 Ad hoc plan cache bloat . . . 46
11. Version Features and Plan Control . . . 47
11.1 Intelligent Query Processing by version . . . 47
11.2 Raising compatibility level safely . . . 47
11.3 Plan control tools . . . 48
12. A Tuning Workflow with Worked Examples . . . 49
12.1 Case 1: fast for one customer, slow for another . . . 49
12.2 Case 2: report spills and takes minutes . . . 50
12.3 Case 3: a query that ignores its index . . . 50
12.4 One-page checklist . . . 50
Part II: Wait Types . . . 51
13. Wait Statistics Fundamentals . . . 51
13.1 What a wait is . . . 51
13.2 The three families of waits . . . 52
13.3 Where to read waits . . . 52
13.4 How to read wait statistics properly . . . 53
13.5 Query Store wait categories . . . 54
14. Wait Type Encyclopedia . . . 56
14.1 CPU and scheduling waits . . . 56
14.2 Parallelism waits . . . 57
14.3 Memory waits . . . 59
14.4 Data file and disk I/O waits . . . 61
14.5 Transaction log waits . . . 63
14.6 Lock waits (LCK_M_*) . . . 65
14.7 Latch waits . . . 68
14.8 Network, client and linked-server waits . . . 70
14.9 Batch mode and columnstore waits . . . 71
14.10 Availability groups, mirroring and replication waits . . . 72
14.11 Preemptive and external waits . . . 74
14.12 Transactions, DTC and miscellaneous engine waits . . . 76
14.13 Service Broker, CLR, full-text, FILESTREAM, in-memory OLTP and Query Store . . . 77
14.14 Background, idle and benign waits . . . 78

14.15 Internal-use and undocumented waits . . . 79
14.16 Azure SQL and Managed Instance notes . . . 80
15. Wait-Based Troubleshooting Playbooks . . . 81
15.1 First step: group waits by resource . . . 81
15.2 CPU pressure . . . 81
15.3 Data I/O pressure . . . 82
15.4 Transaction log bottleneck . . . 82
15.5 Blocking and locking . . . 82
15.6 Memory: grants and compilation . . . 82
15.7 tempdb allocation contention . . . 82
15.8 Network and client . . . 83
15.9 Availability group latency . . . 83
15.10 Connecting plans to waits . . . 83
Appendix A: Operator Quick Reference . . . 85
Appendix B: Diagnostic Toolkit . . . 85
Appendix C: Glossary . . . 86
Appendix D: Where to Learn More . . . 87

How to Use This Guide

This guide teaches a DBA to analyse SQL Server execution plans and to understand wait types. Part I explains how plans are made and read, every physical operator, the physical joins in depth, parallelism, memory grants and common fixes. Part II explains wait statistics and gives an encyclopedia of wait types, with playbooks that connect waits to plans.

Chapter	You will learn	Read it when you...
1. How SQL Server Runs a Query	Pipeline, cost-based optimizer, iterator model, row vs batch mode	need the mental model
2. Capturing Plans	Estimated, actual, last-known and live plans; DMVs, Query Store, XEvents	need to get a plan
3. Reading a Plan	Ten-step method, key properties, seeks and scans	must analyse a plan
4. Cardinality and Statistics	How estimates are made and why they fail	see bad estimates
5. Warnings and Red Flags	Every plan warning and the ones without icons	triage a plan quickly
6. Physical Operators	Complete reference of operators with tuning tips	meet an unknown operator
7. Physical Joins	Nested Loops, Merge, Hash, Adaptive in depth	tune joins
8. Parallelism	Exchanges, skew, inhibitors, settings	tune parallel plans
9. Memory Grants and Spills	Grants, feedback, spills, fixes	see spills or RESOURCE_SEMAPHORE
10. Common Patterns	Lookups, SARGability, sniffing, UDFs and more	need a fix recipe
11. Versions and Plan Control	IQP by version, hints, Query Store forcing	control plans
12. Tuning Workflow	A repeatable loop with worked examples	start a tuning task
13. Wait Fundamentals	Thread states, DMVs, scripts, Query Store categories	start with waits
14. Wait Type Encyclopedia	Detailed cards and reference tables by resource	look up a wait
15. Playbooks	Wait-driven troubleshooting and plan-to-wait mapping	diagnose a symptom

A note on completeness and accuracy

SQL Server exposes more than a thousand wait types and many are internal. This guide covers every wait a DBA normally meets, every documented family and how to treat the rest. The operator list follows Microsoft's showplan reference (including Adaptive Join, Optimized Nested Loops and, in SQL Server 2025, Vector Index Seek). Versions and builds differ, so confirm specifics in the documentation for your release.

1. How SQL Server Runs a Query

An execution plan is the **recipe** SQL Server follows to answer your query: which indexes to read, in which order to join tables, how to sort or group, and how much memory to ask for. To read plans well you first need a simple picture of how the recipe is made and used.

1.1 From T-SQL text to results

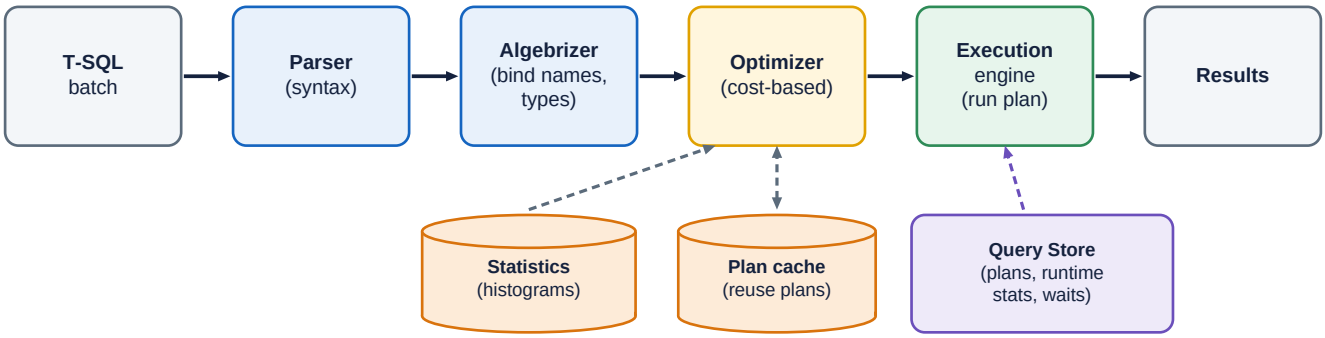


Figure 1.1 - The query processing pipeline and the objects it uses

Stage	What happens	Why a DBA cares
Parse	Checks syntax and builds a parse tree	Syntax errors appear here; no plan yet
Bind (algebrizer)	Resolves object names, columns and data types; builds a logical query tree	Missing objects, permissions and implicit type rules apply here
Optimize	Looks in the **plan cache**; if no plan, explores alternatives using **statistics** and a cost model, then picks the cheapest estimated plan	Bad statistics or odd predicates lead to bad plans. Compile time and CPU are real costs
Execute	Runs the plan operator by operator and returns rows	Actual rows, memory use, spills and waits are measured here
Cache and store	The compiled plan is cached; Query Store (if on) records plans and runtime stats	Plans are reused for the same text and parameters, which causes both speed and parameter sniffing

1.2 The optimizer is cost-based, not time-based

The optimizer does not know how long a query will take. It estimates **row counts** from statistics and converts them into a unit-less **cost** using a model of CPU and I/O. It then chooses the plan with the lowest estimated cost. Every number you see on a plan before the query runs (estimated rows, estimated cost, cost percentages) is a guess, and the plan is only as good as the guesses.

> **Key idea**
> **Cost % in a plan is an estimate, not a measurement.** An operator showing 5% cost can take 95% of the time if its row estimate was wrong. Always compare estimated numbers with **actual** numbers (rows, executions, time, reads).

Optimization levels and early exit

- **Trivial plan:** for very simple queries (for example a single-table lookup on a unique key) there is only one sensible plan, so the optimizer skips cost-based search.
- **Full optimization:** the optimizer explores alternatives in phases, stopping when a plan is 'good enough' or when it runs out of time or memory. The plan's **StatementOptmEarlyAbortReason** shows *TimeOut* or *MemoryLimitExceeded* when it gave up early. Complex queries with many joins often time out and get a suboptimal plan.
- **Cached plans are reused** until something invalidates them: schema change, statistics update, SET option difference, memory pressure, `OPTION (RECOMPILE)`, or a manual cache clear.

1.3 How a plan runs: the iterator (pull) model

A plan is a **tree of operators**. Each operator is a small program with three methods: `Open()`, `GetRow()` and `Close()`. The root asks its child for a row; the child asks its own children, down to a leaf that reads from a table or index. The row then travels back up. In the graphical plan the arrows point **right to left** (the direction rows flow), but execution is driven from the **left** (the root pulls).

Top input of a join (outer) is drawn above; bottom input (inner) below.

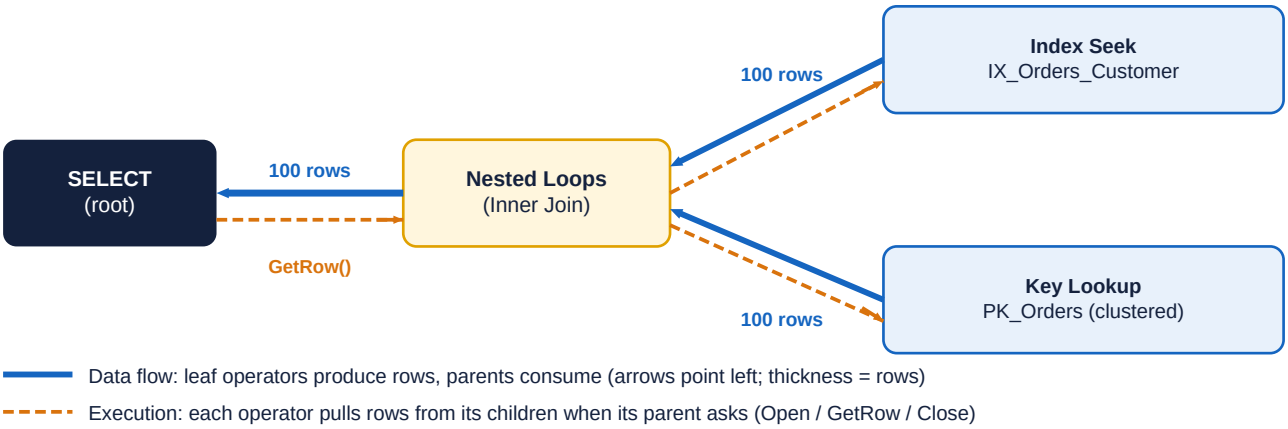


Figure 1.2 - Data flows right to left; execution is pulled from the root

- **Streaming (non-blocking)** operators pass rows along as they get them: Nested Loops, Merge Join, Stream Aggregate, Compute Scalar, Filter, Top.
- **Blocking (stop-and-go)** operators must consume all or part of their input before returning the first row: Sort, Hash Match (build side and aggregates), Eager Spool. They need memory and can spill to tempdb.
- **Early termination:** a Top above a scan stops the scan as soon as it has enough rows. This is called a **row goal**, and it is why a scan under a Top may read only a few rows.
- **Rebinds and rewinds:** an operator on the inner side of a Nested Loops runs once per outer row. *Actual Executions* = number of times its Open() ran. A rebind re-evaluates with new parameters; a rewind reuses the previous result.

1.4 Row mode and batch mode

	Row mode	Batch mode
Rows per call	One row per GetRow()	A batch of up to about 900 rows
Designed for	OLTP-style small result sets	Analytics: large scans, joins and aggregates on many rows
Where used	All rowstore plans (traditionally)	Columnstore indexes (2012+); also rowstore ('batch mode on rowstore', compatibility level 150+)
Operators	All operators	Subset: scans, hash join, hash aggregate, window aggregate, sort, compute scalar, filter, adaptive join and others
Plan clue	Properties: Actual Execution Mode = Row	Properties: Estimated/Actual Execution Mode = Batch
Timing	Operator times include child operators	Operator times are per operator

2. Capturing Execution Plans

There are four kinds of plan, and each answers a different question. Choose the lightest one that answers yours.

2.1 Types of plans

Plan type	What it is	Contains	How to get it
Estimated plan	Compiled plan; the query is **not** run	Estimated rows, cost, estimates only	SSMS Display Estimated Plan; SET SHOWPLAN_XML ON; plan cache; Query Store
Actual plan	The same plan **after** running the query, with runtime counters added	Actual rows, executions, elapsed/CPU time, memory grant, waits, spills, thread data	SSMS Include Actual Plan; SET STATISTICS XML ON; XEvent
Last known actual plan	Last actual plan of a cached query, kept in memory	Actual counters of the last execution	sys.dm_exec_query_plan_stats (SQL Server 2019+, needs the LAST_QUERY_PLAN_STATS setting)
Live plan (in flight)	Plan of a query that is running right now, updated as it runs	Rows so far per operator, progress	SSMS Live Query Statistics; sys.dm_exec_query_statistics_xml

> **An 'actual' plan is not re-optimized**
> The actual plan has the **same operators and the same estimates** as the estimated plan. The difference is that runtime numbers are added. If estimates and actuals disagree, the plan was built on wrong assumptions.

2.2 Capturing in SSMS and with SET options

```
-- Estimated plan (query is NOT executed); run as the only statement in the batch
SET SHOWPLAN_XML ON;
GO
SELECT o.OrderID, c.Name FROM dbo.Orders o JOIN dbo.Customers c ON c.CustomerID = o.CustomerID WHERE
    o.OrderDate >= '2026-01-01';
GO
SET SHOWPLAN_XML OFF;
GO

-- Actual plan plus I/O and time statistics (the query IS executed)
SET STATISTICS XML ON;
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
SELECT o.OrderID, c.Name FROM dbo.Orders o JOIN dbo.Customers c ON c.CustomerID = o.CustomerID WHERE
    o.OrderDate >= '2026-01-01';
SET STATISTICS XML OFF; SET STATISTICS IO OFF; SET STATISTICS TIME OFF;
```

Setting	Returns	Use
SET SHOWPLAN_XML / SHOWPLAN_TEXT / SHOWPLAN_ALL	Estimated plan, query not run	Safe on production for expensive queries
SET STATISTICS XML ON	Actual plan as XML, query run	Full detail; best for analysis
SET STATISTICS PROFILE ON	Actual plan as text rows with Rows and Executes	Quick text view; legacy
SET STATISTICS IO ON	Logical/physical reads, read-ahead, LOB reads, scans per table	Measure data access cost; the best single metric for tuning

Setting	Returns	Use
SET STATISTICS TIME ON	Parse/compile and execution CPU and elapsed time	Measure real time and compile time

2.3 Getting plans from the cache and Query Store

```
-- Top 10 cached statements by CPU, with their plans
SELECT TOP 10
       qs.total_worker_time / 1000 AS total_cpu_ms, qs.execution_count,
       qs.total_worker_time / qs.execution_count / 1000 AS avg_cpu_ms,
       qs.total_logical_reads / qs.execution_count AS avg_reads,
       SUBSTRING(st.text, qs.statement_start_offset / 2 + 1,
                 (CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text) ELSE
                     qs.statement_end_offset END
                  - qs.statement_start_offset) / 2 + 1) AS statement_text,
       qp.query_plan, qs.query_hash, qs.query_plan_hash
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
ORDER BY qs.total_worker_time DESC;

-- Last actual plan of cached queries (SQL Server 2019+)
ALTER DATABASE SCOPED CONFIGURATION SET LAST_QUERY_PLAN_STATS = ON;
SELECT qps.query_plan FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_query_plan_stats(qs.plan_handle) qps WHERE qs.query_hash = 0x<hash>;

-- A query that is running right now (live plan)
SELECT r.session_id, r.status, r.wait_type, qsx.query_plan
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_query_statistics_xml(r.session_id) qsx WHERE r.session_id = 57;
```

```
-- Query Store: plans, runtime stats and waits for one query
SELECT q.query_id, p.plan_id, p.is_forced_plan, p.query_plan,
       rs.count_executions, rs.avg_duration / 1000.0 AS avg_ms, rs.avg_cpu_time / 1000.0 AS avg_cpu_ms,
       rs.avg_logical_io_reads, rs.avg_query_max_used_memory
FROM sys.query_store_query q
JOIN sys.query_store_plan p ON p.query_id = q.query_id
JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id
WHERE q.query_id = 12345;
```

2.4 Other sources and cautions

Source	Notes
Query Store (SSMS reports: Top Resource Consuming Queries, Regressed Queries, Tracked Queries)	Best for history and comparing plans over time; can force a plan; stores waits per plan
Extended Events: query_post_execution_showplan	Captures actual plans but is **heavy**; use short sessions with strict filters. The lightweight profiling events (for example query_plan_profile) are cheaper
sys.dm_exec_query_profiles	Per-operator live row counts for running queries (needs profiling on; lightweight profiling is on by default in recent versions)
SSMS 'Compare Showplan'	Side-by-side comparison of two saved plans
Azure Data Studio, third-party plan viewers	Alternative visualisations; always keep the raw .sqlplan XML

- **Permissions:** SHOWPLAN on the database for estimated plans; VIEW SERVER STATE (VIEW SERVER PERFORMANCE STATE from SQL Server 2022) and VIEW DATABASE STATE for DMVs.
- **Overhead:** capturing actual plans changes timing slightly, and Live Query Statistics and XEvent plan capture can slow busy systems. Capture on a test copy when you can.

- **Save the XML:** a .sqlplan file is plain XML containing everything. Attach it to tickets so others can open it.
- **Cached plans may be 'compile-time' only:** the plan in cache normally holds estimates, not actuals; actual numbers come from the methods above.

3. How to Read an Execution Plan

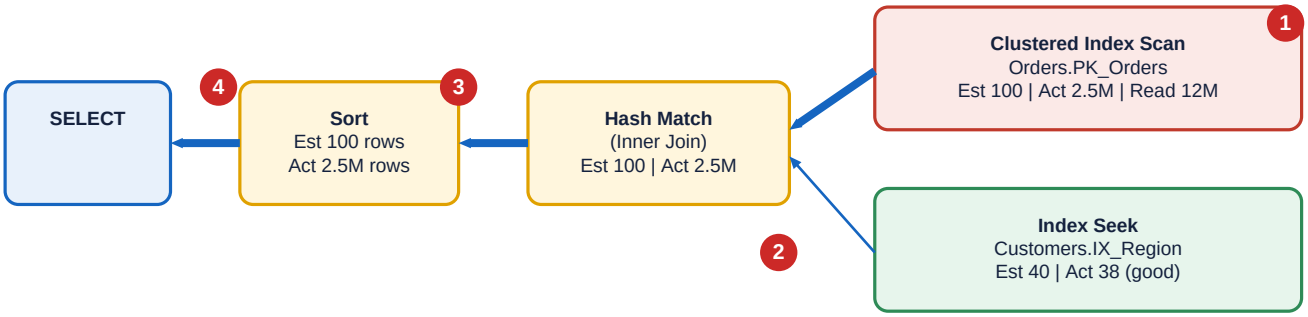
Reading a plan is a skill with a method. Follow the same order every time, and look at **actual work** before cost.

3.1 The ten-step method

Step	What to do	Where to look
1	Read the statement-level facts: compile time, optimization level, early-abort reason, DOP, memory grant, parameters, CE version	Click the root operator (SELECT / INSERT / UPDATE): Properties pane
2	Find where the **time and rows** really are: operators with high actual elapsed/CPU time, or huge actual row counts	Actual Elapsed Time, Actual CPU Time, Actual Rows, arrow thickness
3	Compare **estimated vs actual rows** on every important operator (ratio of 10x or more is a red flag)	Estimated Number of Rows Per Execution vs Actual Number of Rows (per execution)
4	Follow the data flow right to left and note where row counts explode or collapse	Arrow widths; row counts on each operator
5	Check data access: seek or scan? lookups? is the predicate a seek predicate or a residual predicate?	Seek Predicates, Predicate, Number of Rows Read vs Actual Rows
6	Check joins: join type vs row counts; inner executions; many-to-many; hash spills	Nested Loops Actual Executions, Merge Many-to-Many, Hash warnings
7	Check sorts, spools, spills and memory grants	Warnings on Sort / Hash; Memory Grant Info on root; Spool operators
8	Check parallelism: DOP, skew between threads, Gather/Repartition, reasons for serial plans	Actual Rows per Thread; NonParallelPlanReason
9	Read warnings and missing-index hints; judge them critically	Yellow triangles; green 'Missing Index' text
10	Cross-check with STATISTICS IO/TIME and the WaitStats in the plan	Messages tab; root operator WaitStats

3.2 An annotated example

The figure shows a typical bad plan. Read it with the method above.



1. Scan reads 12M rows to return 2.5M: no useful index on the filter column; add an index or fix a non-SARGable predicate.
2. Estimate 100 vs actual 2.5M (25,000x): stale statistics, parameter sniffing or a bad predicate estimate.
3. Hash Match got a memory grant sized for 100 rows, so it spilled to tempdb (Hash Warning).
4. Sort also spilled; the same wrong estimate gave it a tiny memory grant.

Figure 3.1 - A plan with a wrong estimate that causes a scan, a hash join and two spills

Diagnosis in one sentence: the optimizer expected about 100 orders but 2.5 million qualified, so it picked a scan feeding a hash join and a sort with tiny memory grants, which then spilled to tempdb. The root cause is the **estimate**, not the join or the sort. Fix the estimate (statistics, parameter handling, predicate shape) and the plan, memory grant and spills usually fix themselves.

3.3 Key operator properties

Click an operator and open the **Properties** pane (F4 in SSMS). The tooltip shows only a subset. These are the fields that matter most:

Property	Meaning	How to use it
Estimated Number of Rows Per Execution / Actual Number of Rows	Optimizer guess vs reality	The most important comparison. For inner sides of loops, compare per execution
Actual Executions (Number of Executions)	How many times the operator ran	Inner side of Nested Loops = outer rows. High executions x per-execution cost = real cost
Number of Rows Read	Rows examined before the residual predicate	If much larger than Actual Rows, the predicate is not being used for seeking
Estimated / Actual Row Size	Bytes per row	Wide rows increase sort/hash memory and spill risk
Estimated Operator Cost, Subtree Cost	Estimated cost units	Use only to understand the optimizer's view
Actual Elapsed / CPU Time (ms)	Measured time for the operator	In row mode the numbers include child operators; in batch mode per operator
Actual I/O Statistics	Reads for this operator (logical, physical, read-ahead)	Shows which operator does the I/O
Seek Predicates / Predicate	Seek keys used to navigate the index / filter applied to rows found	Seek on a leading column good; Predicate only = residual filtering
Ordered, Scan Direction	Whether the operator returns rows in index order	Needed for merge joins, stream aggregates, TOP ORDER BY without sort

Property	Meaning	How to use it
Output List / Defined Values	Columns produced / expressions computed	Wide output lists force key lookups and bigger rows
Estimated / Actual Execution Mode	Row or Batch	Batch mode is usually faster for large analytical scans
Parallel, Actual Rows per Thread	Operator ran in parallel / distribution across threads	Uneven threads = skew
Memory Fractions, Memory Grant Info	Share of the grant used by each operator; requested, granted, used, wait time	Detect over- and under-sized grants
Warnings	Spill, implicit conversion, missing statistics, no join predicate and others	Always read them (chapter 5)
WaitStats	Top waits for this execution (actual plans)	Connects the plan to wait types (Part II)

3.4 Statement-level information on the root operator

Property	Why it matters
Cached Plan Size, Compile Time, Compile CPU, Compile Memory	High compile cost on frequently run queries suggests parameterisation or plan reuse problems
Statement Optimization Level / Early Abort Reason	TRIVIAL or FULL; TimeOut means the search stopped early on a complex query
Degree of Parallelism, Non Parallel Plan Reason	Why the plan runs serial or parallel
Cardinality Estimation Model Version	70 = legacy CE; 120 and above = newer CE; depends on compatibility level
Parameter List: Compiled value vs Runtime value	Parameter sniffing: plan built for one value, run with another
Set Options	ARITHABORT, ANSI_NULLS and others create separate cache entries; mismatches explain 'fast in SSMS, slow in the app'
Query Hash / Query Plan Hash	Group similar queries; detect plan changes
Memory Grant Info	Requested, granted, max used and grant wait time
Missing Indexes	Optimizer's suggestion; evaluate before creating
Wait Stats	Waits observed by this query, in ms and counts
Query Time Stats (CPU, Elapsed)	Total runtime of the statement

3.5 Seeks, scans and 'Number of Rows Read'

A **seek** navigates the B-tree to a starting key and reads only the rows needed; a **scan** reads the whole index or heap (or a range of it). Seek is not automatically good and scan is not automatically bad: a scan of a small table is fine, and a seek returning most of a table is expensive. What matters is the amount of work compared with the rows you need.

Observation	Meaning	Action
Index Seek with Seek Predicate on the first key column	Efficient navigation	Usually good

Observation	Meaning	Action
Seek with a Predicate (residual) that removes many rows	Index navigates by one column only; later conditions are filtered row by row	Reorder or extend index so more predicates become seek predicates
Scan, Number of Rows Read much larger than Actual Rows	Reading rows only to throw them away	Add an index on the filter columns; make predicate SARGable
Seek returning millions of rows	Range is too wide to be selective	Consider whether a scan/hash plan or a different design is better
Key Lookup executed many thousands of times	Index does not cover the query	Add INCLUDE columns or narrow the select list

SARGable predicates

A predicate is **SARGable** (Search ARGument able) when SQL Server can use it to seek. Functions on the indexed column (`WHERE YEAR(OrderDate) = 2026`), leading wildcards (`LIKE '%abc'`) and implicit conversions on the column side usually make it non-SARGable. Rewrite as `OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'`.

4. Cardinality Estimation and Statistics

Almost every bad plan starts with a bad **cardinality estimate** (the number of rows the optimizer expects). If you can explain why an estimate is wrong, you can usually fix the plan. This chapter shows where estimates come from and how they fail.

4.1 Where estimates come from

For every column used in a predicate or join, SQL Server keeps **statistics**: a **histogram** (up to 200 steps describing the value distribution of the first key column), a **density vector** (how unique each prefix of the key columns is), and the row count. The **cardinality estimator (CE)** combines these into a row estimate for each operator.

Statistics part	What it holds	Used for
Header	Rows, rows sampled, last update, steps	Staleness and sample size
Density vector	1 / number of distinct values for column prefixes	Equality estimates when the value is unknown (variables, joins), GROUP BY estimates
Histogram	RANGE_HI_KEY, RANGE_ROWS, EQ_ROWS, DISTINCT_RANGE_ROWS, AVG_RANGE_ROWS	Estimates for specific values and ranges

```
DBCC SHOW_STATISTICS ('dbo.Orders', 'IX_Orders_OrderDate');      -- header, density vector, histogram

SELECT s.name AS stat, sp.last_updated, sp.rows, sp.rows_sampled, sp.modification_counter
FROM sys.stats s CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE s.object_id = OBJECT_ID('dbo.Orders');

UPDATE STATISTICS dbo.Orders IX_Orders_OrderDate WITH FULLSCAN;  -- most accurate; costs I/O
```

4.2 How estimates are built

Predicate	Typical estimate method
Column = literal value present in the histogram	EQ_ROWS of the matching step (accurate)
Column = literal value inside a histogram step	AVG_RANGE_ROWS of that step
Column = variable or unknown value	Density vector: rows x (1 / distinct values), i.e. an average, not value-specific
Column > or < variable	A fixed guess of a fraction of the table (for example about 30%)
Parameter in a stored procedure or sp_executesql	Value at first compile is 'sniffed' and used with the histogram
Several predicates ANDed	Legacy CE: multiply selectivities (assumes independence). New CE (compatibility 120+): exponential backoff, which dampens the product for correlated columns
Join	Join selectivity from histograms/densities of the join columns and a containment assumption
Value above the highest histogram step (ascending keys such as dates and identities)	Newer CE extrapolates; legacy CE assumed about one row. Update statistics frequently on such columns
Table variable	1 row before SQL Server 2019. From compatibility level 150, 'deferred compilation' uses the actual row count at first compile
Multi-statement table-valued function	Fixed guess (1 or 100 rows by version). Compatibility 140+ 'interleaved execution' uses the real row count

The CE model is chosen by the database compatibility level: 120 and above use the newer estimator. In SQL Server 2022 and later, CE feedback can adjust model assumptions per query.

4.3 When statistics update

- **Auto update statistics** (default on) refreshes a statistic when enough rows have changed. The classic threshold is 500 rows + 20% of the table. With compatibility level 130 and above the threshold is dynamic and much lower for large tables (about the square root of 1,000 x rows), so big tables refresh sooner.
- **Auto update async** lets the query run with old statistics while the update happens in the background, which avoids compile waits but allows a bad plan once.
- **Sampling:** default sampling may miss skew in very large tables. Use FULLSCAN or a higher sample, or persisted sample percent, for critical tables.
- **Rebuilding an index** updates its statistics with a full scan; **reorganizing does not**.
- **Filtered statistics, multi-column statistics and incremental statistics** (partitioned tables) are tools for skewed or correlated data.

4.4 Why estimates go wrong and what to do

Cause	How it appears in the plan	Fix
Stale or low-sample statistics	Estimates far from actuals on many operators of one table	UPDATE STATISTICS (FULLSCAN or higher sample); update more often
Parameter sniffing	Compiled value differs from runtime value; good for one value, terrible for another	OPTION (RECOMPILE), OPTIMIZE FOR, PSP optimization, Query Store hints, procedure design, plan forcing
Local variables	Estimate uses density average, ignores the value	Use parameters, OPTION (RECOMPILE), or dynamic SQL
Correlated columns	Row estimate too low (independence assumption) after several ANDed predicates	Multi-column statistics, filtered statistics, new CE, redesign
Skewed data distribution	Averages hide hot values	Higher sample, filtered stats, split queries, PSP optimization
Functions or expressions on columns	Optimizer cannot use the histogram; uses a guess	Rewrite predicate SARGable, computed column with index
Implicit conversions	CONVERT_IMPLICIT on the column side; guessed estimates and scans	Match data types between column and parameter
Table variables (old versions), MSTVFs	Estimate of 1 or 100 rows regardless of data	Temp tables, inline TVFs, compatibility level 140/150+
Ascending key beyond the histogram	Estimate of about 1 row for newest data	Frequent statistics updates; newer CE
Linked servers and remote queries	Remote data estimates are crude	Move data local, or filter remotely
Complex expressions, many joins	Errors multiply through the plan	Break the query into steps using temp tables; simplify; index the join columns
Optimizer timeout	Early-abort reason TimeOut; plan not fully explored	Simplify the query or reduce joins; check for plan guides and hints

Rule of thumb

An estimate that is off by **less than 10x** rarely changes the plan shape. An error of **100x or more** almost always does, because it changes join types, memory grants, spools and parallelism choices.

5. Plan Warnings and Red Flags

SSMS shows warnings as a yellow triangle on an operator or the root. Some are urgent and some are noise. This chapter explains each one and lists red flags that have no icon at all.

5.1 Warnings shown in the plan

Warning	Meaning	What to do
Missing index	The optimizer thinks an index would reduce cost; shown as green text under the statement	Treat as a hint, not an order. It ignores existing similar indexes, may suggest very wide includes and is based on one query. Combine with index usage data and test
Implicit conversion (PlanAffectingConvert; ConvertIssue = CardinalityEstimate or SeekPlan)	A data type conversion on a column prevents a seek or distorts estimates (for example varchar column compared to nvarchar parameter)	Make the parameter or literal match the column type; fix application parameter types
Columns with no statistics	The optimizer had to guess selectivity	Enable auto-create statistics or create them; check whether the table is a temp or read-only database
No join predicate	A join without ON conditions, a cross product	Usually a missing condition. If intended, ignore; otherwise fix the query
Sort warning / Hash warning (spill to tempdb)	The operator needed more memory than it was granted and wrote data to tempdb (with a spill level and number of spilled threads)	Fix the estimate, reduce rows or width before the sort/hash, add an index to avoid the sort, review memory grant feedback
Excessive memory grant (grant warning)	Granted much more than used; wastes memory and can cause RESOURCE_SEMAPHORE waits for others	Fix estimates; MAX_GRANT_PERCENT hint; memory grant feedback; reduce row width
Memory grant increase or 'used more than granted'	Underestimated need; spills likely	Fix the estimate; update statistics; avoid string-heavy sorts
Unmatched indexes	A filtered index could not be used because of parameterisation	Add OPTION (RECOMPILE) or use a literal that matches the filter
Spill in an exchange (parallelism)	Rows buffered in exchange spilled to tempdb; often an intra-query parallel deadlock	Check order-preserving exchanges and skew; reduce DOP; adjust indexing
Plan affecting convert on join keys	Join columns of different types	Align column types in the schema
Optimizer timeout / memory limit	Search stopped early; plan may be poor	Simplify; break up the query; avoid extremely long IN lists or many joins
Row goal	TOP, EXISTS or FAST n made the optimizer favour plans that return the first rows fast	Compare behaviour when many rows are really needed; use DISABLE_OPTIMIZER_ROWGOAL hint if it hurts
Parameter 'sniffed' value differs	Compiled and runtime parameter values differ	See parameter sniffing options (chapter 10)

5.2 Red flags without an icon

Red flag	Why it hurts	Typical cause
Estimated rows 1 or 100 and actual rows in the millions	Plan built for a tiny set	Table variable, MSTVF, stale stats, sniffed parameter
Key Lookup executed hundreds of thousands of times	Random I/O per row	Non-covering index chosen for a wide range
Clustered Index Scan reading many more rows than returned	Wasted reads	Missing index or non-SARGable predicate
Nested Loops with a huge outer input	Inner side repeated for each outer row	Underestimated outer row count
Hash Match with a spill	tempdb I/O for hashing	Underestimated build input or too-small grant
Sort of millions of rows	Memory and spill risk, blocking	No index in required order; ORDER BY on many wide columns
Eager Spool in a read plan or Table Spool inside a loop	Builds temporary storage per execution	Optimizer needs an index it does not have, or Halloween protection in updates
Merge Join many-to-many = True	Uses a tempdb worktable and rewinds	Non-unique join keys on both sides
Parallel plan with all rows on one thread	No real parallelism (skew)	Bad data distribution or a serial region
Compute Scalar calling a scalar UDF	Runs the function per row, often serial and slow	Non-inlined scalar user-defined function
Table-valued function operator with large output	Multi-statement TVF materialised in tempdb with a fixed estimate	MSTVF in a join
Plan changes between runs of the same query	Unstable performance	Parameter sensitivity, statistics changes, hints, compatibility changes
Large difference between 'Number of Rows Read' and 'Actual Rows'	Residual filtering	Index not aligned with predicate
Late Filter operator after expensive joins	Filtering applied late	Predicate not pushed down because of expressions or outer-join semantics

5.3 Missing indexes: use with care

- The suggestion lists equality columns first, then inequality columns, then INCLUDE columns. It does not consider column order for sort needs, existing overlapping indexes, write cost or index size.
- Check `sys.dm_db_missing_index_group_stats` for how often and how much benefit, and consolidate several suggestions into one carefully designed index.
- Create the index in a test environment, re-capture the plan and compare reads, CPU and writes.

```
SELECT TOP 20 ROUND(gs.avg_total_user_cost * gs.avg_user_impact * (gs.user_seeks + gs.user_scans), 0)
    AS score,
       d.statement AS table_name, d.equality_columns, d.inequality_columns, d.included_columns,
           gs.user_seeks
FROM sys.dm_db_missing_index_group_stats gs
JOIN sys.dm_db_missing_index_groups g ON g.index_group_handle = gs.group_handle
JOIN sys.dm_db_missing_index_details d ON d.index_handle = g.index_handle
ORDER BY score DESC;
```

6. Physical Operators: Complete Reference

This chapter explains every operator you can meet in a SQL Server plan, grouped by purpose. **Physical** operators do the work; **logical** operators describe the relational idea (for example 'Inner Join') that the optimizer maps onto a physical operator (for example Hash Match). Each table row says what the operator does, why it appears, and what to check. Joins are explained in full in chapter 7 and parallelism operators in chapter 8.

6.1 How to use this reference

- Find the operator name on the plan tooltip, then locate its row here.
- The **'watch'** column is a short checklist; the red flags in chapter 5 and the patterns in chapter 10 give fuller fixes.
- Operators marked *(batch)* run in batch mode when the plan uses columnstore or batch mode on rowstore; most others run in row mode.
- Operators that exist only in some versions are marked, for example *SQL Server 2025*.

6.2 Reading data: scans, seeks and lookups

Operator	What it does	When you see it	What to watch / how to tune
Table Scan	Reads every row of a **heap** (table without a clustered index), in allocation order	Heap with no usable nonclustered index, or when most rows are needed	Large heap scans; forwarded records (extra reads); predicate evaluated per row. Fix: index, clustered index, predicate rewrite
Clustered Index Scan	Reads the leaf level of the clustered index (the table) in index or allocation order	No seekable index for the predicate, or most rows required, or small table	Compare Number of Rows Read with Actual Rows; check Ordered; consider an index on filter columns or a covering index
Clustered Index Seek	Navigates the B-tree to the first matching key then reads a range; uses Seek Predicates	Predicate on the leading clustered key column(s)	A seek can still read millions of rows; check Predicate (residual) and row counts; singleton seeks inside loops execute many times
Index Scan (nonclustered)	Reads all leaf pages of a nonclustered index	Covering index but no useful seek predicate (e.g. unfiltered COUNT, wide range)	Narrower than a table scan so often acceptable; consider a better key order if the predicate is selective
Index Seek (nonclustered)	Navigates a nonclustered index using Seek Predicates	Selective predicate on the index's leading columns	Often followed by a Key Lookup. Check lookup count; multiple seek predicates mean an IN/OR list expanded into ranges
Key Lookup (Clustered)	For each row found in a nonclustered index, fetches missing columns from the clustered index using the clustering key	Nonclustered index does not contain all required columns; always paired with Nested Loops	Executions x random reads. Add INCLUDE columns, narrow SELECT list, or accept if rows are few. Large counts push the optimizer to a scan
RID Lookup	Same as Key Lookup but for a heap using a row identifier (file:page:slot)	Nonclustered index on a heap	Add the missing columns to the index or create a clustered index
Constant Scan	Produces a fixed number of rows (zero or more columns)	VALUES lists, IN lists, scalar aggregates, dummy rows for joins, partition ranges	Normally trivial. Large IN lists become many constant rows and ranges; use a temp table or table-valued parameter
Merge Interval	Combines overlapping search ranges into minimal ranges for an index seek	Multiple OR or IN predicates on the same column	Part of multi-range seeks; usually cheap
Sort (for seeks)	Sorts the generated seek keys so the seek reads in index order	Seek driven by a list of keys	Usually minor

Seek and scan details worth knowing

- **Seek predicate vs predicate:** the seek predicate navigates the B-tree. A **Predicate** on a seek or scan is a residual filter applied to each row read. Rows read minus rows returned is wasted work.
- **Ordered scan vs allocation-order scan:** if the plan does not need order, the engine may scan in allocation order (faster, but can read pages twice or skip pages under concurrent page splits at lower isolation levels). Ordered = True means rows come out in index key order.
- **Forward / backward scan:** the optimizer can read an index in reverse to satisfy ORDER BY DESC without a sort.
- **Prefetching:** Nested Loops with prefetch issues asynchronous reads for upcoming lookups; it shows as WITH PREFETCH or ORDERED PREFETCH on the join and raises I/O throughput but can hold locks longer.
- **Partition elimination:** for partitioned tables look for the Actual Partition Count, and Seek Predicates on the partition function to confirm only needed partitions are read.

- **Key Lookup tipping point:** as a rule of thumb the optimizer prefers a scan once lookups would touch roughly a quarter to a third of the pages in the table. Many lookups cost one random page read each.

6.3 Columnstore and batch mode

Operator	What it does	When you see it	What to watch / how to tune
Columnstore Index Scan (batch)	Reads compressed column segments of a columnstore index (clustered or nonclustered); applies pushed-down predicates and aggregates	Analytic queries on columnstore; also batch mode on rowstore may show 'Index Scan' in batch mode	Segment/rowgroup elimination (check segment reads and skips in STATISTICS IO), delta store rows, ordered columnstore, predicate pushdown. Single-row lookups are poor
Window Aggregate (batch)	Evaluates window functions (SUM() OVER, LAG, ranking) in batch mode	Window functions in plans eligible for batch mode	Usually much faster than Segment + Sequence Project + Window Spool in row mode
Batch Hash Table Build	Builds a batch hash table for a memory-optimized columnstore; SQL Server 2012 only	Legacy	Not used in newer versions
Adaptive Join (batch)	Defers the choice of Hash or Nested Loops until the build input is read	Batch mode plans, compatibility 140+/150+	See chapter 7

6.4 Join operators (summary)

Operator	What it does	When you see it	What to watch / how to tune
Nested Loops	For each outer (top) row, runs the inner (bottom) input and returns matches	Small outer input and an indexed inner side	Inner executions = outer rows; big outer = trouble. Optimized/prefetch variants exist
Merge Join	Walks two inputs sorted on the join keys in lock-step	Large inputs already ordered by an index; equality joins	Many-to-many uses a tempdb worktable; injected Sorts; parallel merge needs order-preserving exchanges
Hash Match (join)	Builds a hash table from the top (build) input, probes with the bottom input	Large unsorted inputs, no helpful indexes	Memory grant, spills, bad build/probe choice caused by estimates
Adaptive Join	Picks Nested Loops or Hash at run time based on a row threshold	Batch mode, estimates uncertain	Check Actual Join Type and threshold; grant is sized for the hash option

The logical join types (Inner, Left/Right Outer, Full Outer, Left/Right Semi, Left/Right Anti Semi, Cross) and exactly which physical operators implement which are listed in section 7.6.

6.5 Aggregation, sorting, ranking and windowing

Operator	What it does	When you see it	What to watch / how to tune
Stream Aggregate	Computes aggregates over input sorted by the GROUP BY columns; or a scalar aggregate with no grouping	Input already ordered by an index, or scalar COUNT/SUM/MAX	Cheap and streaming. If a Sort sits below it, consider an index in group order
Hash Match (Aggregate)	Builds a hash table keyed by group columns and accumulates aggregates; blocking	Unsorted input with many groups	Memory grant and spills; estimate of group count drives the grant. In parallel plans it appears as a local (Partial) and a global aggregate
Partial Aggregate	Local aggregation before an exchange to reduce rows (logical)	Parallel plans	Implemented by Hash Match; reduces data repartitioned
Sort	Sorts all input rows; blocking; may remove duplicates (Distinct Sort)	ORDER BY, merge join input, stream aggregate input, index build, DISTINCT	Memory grant and tempdb spill; wide rows; fix by index in the required order or fewer columns/rows
TopN Sort	Returns only the first N rows of a sort using a small bounded buffer	TOP (N) ... ORDER BY without a supporting index	Efficient for small N; large N falls back to a full sort
Top	Returns the first N rows or percent, optionally WITH TIES	TOP, OFFSET/FETCH, row goals	Creates a row goal that shapes the plan; with an index order it stops reading early
Distinct / Distinct Sort / Flow Distinct	Remove duplicates (Flow Distinct streams unique rows as they arrive)	DISTINCT, UNION	Often implemented by Hash Match, Sort or Stream Aggregate. DISTINCT added to hide join duplicates is a code smell
Segment	Divides sorted input into groups and flags the first row of each	Window functions with PARTITION BY (row mode)	Requires sorted input; paired with Sequence Project
Sequence Project	Computes ranking and row-number values over segments	ROW_NUMBER, RANK, DENSE_RANK, NTILE (row mode)	Needs a Sort unless an index provides the order; compare with batch mode Window Aggregate
Window Spool	Expands each row into its window frame; may spool to disk or memory	Aggregate window functions with ORDER BY frames	The default frame is RANGE UNBOUNDED PRECEDING, which uses an **on-disk** spool and is slow; write ROWS BETWEEN ... for the fast in-memory spool
Window Aggregate	Batch mode window function evaluation	SQL Server 2016+ batch mode	Prefer batch mode eligibility (columnstore or compat 150+)

6.6 Row shaping: expressions, filters, asserts and unions

Operator	What it does	When you see it	What to watch / how to tune
Compute Scalar	Calculates an expression for each row (arithmetic, CASE, function calls)	Almost every plan	Cost shown is usually 0 but the work may be deferred or hidden: a scalar UDF here runs once per row and can force a serial plan. Inline scalar UDFs (SQL Server 2019+ can inline automatically)
Filter	Removes rows that fail a predicate not pushed into a scan or seek	HAVING, residual conditions, startup expression predicates (IF-like branches for optional parameters)	A Filter late in the plan means rows were produced and then discarded. Startup filters show rebinds without scanning
Assert	Verifies a condition and raises an error if it fails	CHECK constraints, foreign keys in modifications, scalar subqueries that must return one row	A Stream Aggregate + Assert pair means 'subquery must return at most one row'; ensure uniqueness to avoid cost
Concatenation	Appends several input streams (UNION ALL)	UNION ALL, partitioned views, OR rewritten as unions	Cheap; inputs run one after another
Union (logical)	Concatenation plus duplicate removal	UNION	Implemented with Concatenation plus Sort Distinct, Hash Match (Union) or Merge Join (Union). Prefer UNION ALL if duplicates are impossible
Sequence	Runs inputs in order and returns rows of the last input	Wide update plans, multi-statement TVF, graph SHORTEST_PATH	Appears in modification plans with several targets
Switch	Chooses one input based on an expression	Partitioned view inserts, some cursor plans	Rare
Row Count Spool	Counts rows and returns empty rows to test existence	Semi/anti joins, NOT IN, existence checks	Prevents the inner side from being re-run; can indicate NOT IN null handling
Bitmap / Bitmap Create	Builds a compact bit filter from one side of a hash join and applies it on the other side early	Parallel hash joins (star-join style)	Probe predicate like PROBE(Bitmap) removes rows before the exchange; good thing

6.7 Spools: temporary storage inside a plan

A **spool** saves an intermediate result in tempdb (or memory) so it can be reused. Spools protect correctness or avoid repeated work, but they cost tempdb I/O.

Operator	What it does	When you see it	What to watch / how to tune
Table Spool (Lazy)	Stores rows as the parent asks for them; later rewinds read from the spool instead of re-running the child	Inside the inner side of Nested Loops when the same rows are requested repeatedly; recursive CTE anchors	Check Rewinds vs Rebinds. Many rebinds mean the spool is rebuilt often

Operator	What it does	When you see it	What to watch / how to tune
Table Spool (Eager)	Reads **all** input first, then returns rows; blocking	Halloween protection in updates; some subquery rewrites	In SELECT plans an Eager Spool usually means the optimizer lacks an index; in updates it protects against reading rows it has already modified
Index Spool (Eager/Lazy)	Builds a temporary index on the rows, then seeks into it	Correlated subqueries with no suitable index on the inner table	A strong hint that a permanent index would help; the spool is rebuilt per execution
Row Count Spool	Only counts rows	Existence tests	Cheap
Window Spool	Stores window frame rows	Window aggregates (row mode)	See 6.5
Common subexpression spool	Shares one result among multiple consumers	Queries that reference the same expression twice (for example multi-way UNION or recursive plans)	Check tempdb usage

6.8 Parallelism operators

Operator	What it does	When you see it	What to watch / how to tune
Parallelism (Distribute Streams)	Takes one input stream and splits it into several output streams for parallel threads	Start of a parallel region	Partition type (Hash, Round Robin, Broadcast, Demand, Range)
Parallelism (Repartition Streams)	Redistributes rows between sets of threads, for example by join key	Between parallel operators that need different partitioning	Costly; skew when one thread gets most rows; order-preserving versions can deadlock
Parallelism (Gather Streams)	Merges several streams into one; end of a parallel region	Before a serial operator or the final result	Order-preserving (merging) gathers wait for all inputs; a late Gather means a large serial zone
Broadcast	Sends all rows to every consumer (logical)	Small input joined to a big parallel input	Copies the input once per thread
Branch / Segment Repartition	Logical types of repartition at region boundaries	Parallel plans	Informational

Chapter 8 explains exchanges, partitioning types, skew and the wait types they produce.

6.9 Data modification operators

Operator	What it does	When you see it	What to watch / how to tune
Table Insert / Clustered Index Insert / Index Insert	Insert rows into a heap, clustered index or nonclustered index	INSERT, SELECT INTO, indexes maintained by DML	Number of indexes touched; minimal logging requires TABLOCK and conditions; page splits; Index Insert per nonclustered index
Table Update / Clustered Index Update / Index Update	Modify existing rows in place or via delete + insert	UPDATE, MERGE	Updates to key columns become Split + Sort + Collapse + Insert/Delete sequences; keep updated columns out of keys
Table Delete / Clustered Index Delete / Index Delete	Delete rows	DELETE, MERGE	Many indexes mean many delete operators (wide plan); batch large deletes
Table Merge / Clustered Index Merge / Columnstore Index Merge	Apply a merge stream of insert/update/delete actions	MERGE statement	Check ActionColumn logic; MERGE has had plan bugs in older versions, test carefully
Columnstore Index Insert/Update/Delete	Modify a columnstore index (delta store, delete bitmap)	DML on columnstore tables	Large batch inserts (102,400+ rows) go directly to compressed rowgroups
Insert / Update / Delete (logical)	Logical modification operators	Any DML	Implemented by the physical forms above
Split	Splits an update into a delete and an insert (logical and physical)	Updates that change key columns or unique index values	Part of 'wide' update plans
Collapse	Combines adjacent delete+insert of the same key into a single update	After Split + Sort	Reduces index maintenance
Online Index Insert	Index operation done online	ALTER INDEX ... ONLINE = ON, CREATE INDEX ... ONLINE	Shows that the table stays available during the build
Foreign Key References Check	Checks referential integrity in place when a key has more than 253 referencing foreign keys	Tables with huge numbers of foreign keys	Rare
Inserted Scan / Deleted Scan	Read the inserted or deleted pseudo-tables in a trigger	AFTER triggers, OUTPUT clauses	Triggers add hidden cost to every statement
Put	Writes data to an external table (PolyBase / CETAS)	CREATE EXTERNAL TABLE AS SELECT	Targets Hadoop (2016-2019) or Azure storage / S3-compatible (2022+)

Narrow and wide update plans, Halloween protection

- A **narrow (per-row) plan** updates the clustered index and each nonclustered index in one operator per row. A **wide (per-index) plan** processes one index at a time: Split, Sort, Collapse and then an Index Update/Insert/Delete operator per index. Wide plans are chosen for large updates and give better I/O patterns.
- **Halloween protection** prevents an update from reading rows it has already changed (and changing them again). SQL Server adds an Eager Spool, a Sort or a blocking operator between the read and the write when needed. A scan on one index plus updates on the same index is the classic case.
- **Foreign keys and constraints** add **Assert** operators and joins to the referenced tables. Missing indexes on foreign key columns turn these checks into scans.

- **Triggers** run after the statement and appear as extra plan sections with Inserted/Deleted Scans. **Indexed views** are maintained inside the same plan.

6.10 Remote, function and special-purpose operators

Operator	What it does	When you see it	What to watch / how to tune
Remote Query / Remote Scan / Remote Index Scan / Remote Index Seek	Send a query to, or read an object from, a linked server or external source	Four-part names, OPENQUERY, OPENROWSET, distributed queries	Cost estimates are poor; pull less data (filter remotely), use OPENQUERY to push predicates, or cache locally. Waits: OLEDB, MSQL_DQ
Remote Insert / Update / Delete	Modify a remote object	DML against linked servers	Distributed transactions (MSDTC) add overhead
Async Concat	Opens several remote inputs at once and returns rows as they arrive	Distributed partitioned views	Remote only
Table-valued function	Evaluates a T-SQL multi-statement TVF, CLR TVF or XML reader and stores results in tempdb	Calls to TVFs and the XML nodes() method	Fixed row estimates; replace MSTVF with inline TVF; for XML use proper indexes or shred once
UDX (extended operators)	Implements XQuery, XML, FOR XML and other extensions	XML methods, FOR XML	Can be CPU heavy; avoid repeated XML shredding
Parameter Table Scan	Reads the rowset created by INSERT ... EXEC	INSERT ... EXEC procedure	Serial plans; large result sets are slow
Log Row Scan	Scans the transaction log	Internal operations such as change tracking or log-based reads	Rare in user plans
Vector Index Seek	Uses a vector index to find approximate nearest-neighbour rows (SQL Server 2025)	Vector similarity search queries	Check the number of candidates and the distance metric; new in 2025
Intrinsic	Calls an internal T-SQL function	Built-in functions	Language element

6.11 Cursor operators

Plans for cursor operations show the cursor machinery. Physical operators describe *how* the cursor is implemented; logical operators describe *properties*.

Operator	What it does	When you see it	What to watch / how to tune
Dynamic	Cursor that sees all changes made by others	DYNAMIC cursors	Most expensive; rows can move while scrolling
Keyset	Cursor that sees updates but not inserts by others	KEYSET cursors	Keys stored in tempdb
Snapshot	Cursor with a static copy of the data	STATIC / INSENSITIVE cursors	Copies all rows to tempdb
Population Query	Fills the work table when the cursor opens	Keyset/snapshot cursors	Cost happens at OPEN
Fetch Query	Retrieves rows on FETCH	All scrollable cursors	Row-by-row; avoid for set-based work
Refresh Query	Fetches current values for rows in the fetch buffer	Keyset/dynamic cursors	Extra I/O

Operator	What it does	When you see it	What to watch / how to tune
Asynchronous, Optimistic, Primary, Read Only, Scroll Locks, Secondary, Synchronous (logical)	Describe cursor concurrency and population properties	Cursor plans	Informational; prefer set-based T-SQL

6.12 Language elements and legacy operators

Operator	What it does	When you see it	What to watch / how to tune
Result	The root that returns rows to the client	SELECT plans	Root properties hold the statement-level facts
Declare / Assign / Convert / Conditional (If, While)	Variable allocation, assignment, data type conversion, control flow shown in the plan	Procedural T-SQL inside the batch	Language elements, not relational operators; Convert warnings are in chapter 5
Iterator / Language Element (catch-all icons)	Shown when SSMS has no specific icon	Newer or unusual operators	Not an error; read the operator name
Arithmetic Expression	Legacy: computes a new value from row values	Older versions	Not used in newer versions; replaced by Compute Scalar
Bookmark Lookup	Legacy name for key/RID lookup	Older versions	Replaced by Key Lookup and RID Lookup
Cache	Legacy specialised spool storing one row	Older versions	Not used in newer versions

6.13 Logical operators and what implements them

Logical operator	Typical physical implementations
Inner Join, Left/Right Outer Join, Semi, Anti Semi, Cross Join, Full Outer Join	Nested Loops, Merge Join, Hash Match, Adaptive Join (see 7.6)
Aggregate	Stream Aggregate, Hash Match (Aggregate), Window Aggregate, UserDefinedAggregate (CLR)
Distinct, Flow Distinct, Distinct Sort	Hash Match, Stream Aggregate over Sort, Sort (Distinct)
Union	Concatenation + Hash Match (Union) / Sort Distinct / Merge Join (Union)
Insert / Update / Delete / Merge	Table, Clustered Index, Index, Columnstore Insert/Update/Delete/Merge operators
Eager Spool, Lazy Spool	Table Spool, Index Spool
Distribute / Repartition / Gather Streams, Broadcast, Branch and Segment Repartition	Parallelism (exchange iterator)
Cursor logical operators	Dynamic, Keyset, Snapshot, Population Query, Fetch Query, Refresh Query
Cross Apply, Outer Apply	Nested Loops (with outer references) or rewritten to joins
Sort (ORDER BY), TopN Sort	Sort, Top, TopN Sort
Window (OVER clause)	Segment + Sequence Project + Window Spool + Stream Aggregate (row mode), Window Aggregate (batch mode)

7. Physical Joins in Depth

SQL Server has three physical join algorithms (**Nested Loops, Merge Join, Hash Match**) plus **Adaptive Join**, which chooses between two of them at run time. Understanding how each works, what it needs and how it fails lets you predict plans and fix bad ones.

7.1 The big picture

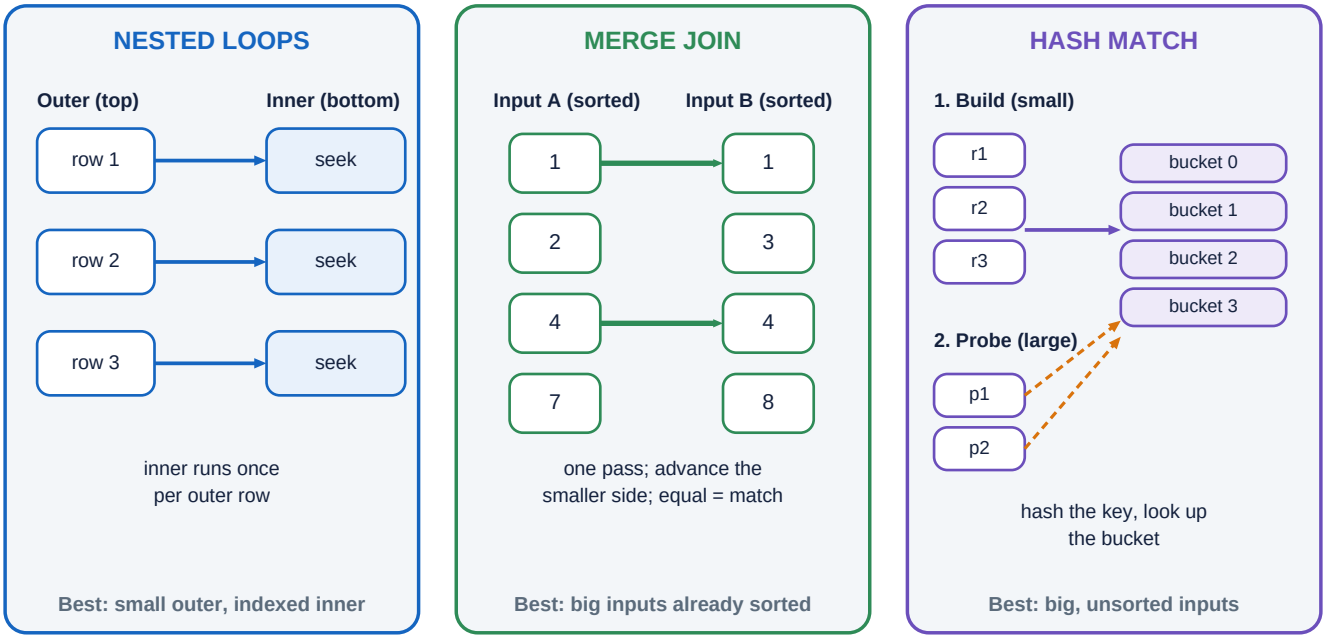


Figure 7.1 - How the three join algorithms work

	Nested Loops	Merge Join	Hash Match
Idea	For each outer row, look up matches in the inner input	Walk two sorted inputs together	Hash the smaller input, probe with the larger
Needs sorted input	No (but an index on the inner side is key)	Yes, on the join keys	No
Join predicates	Any (equality, range, expressions, none)	Equality (plus optional residual)	Equality (plus optional residual)
Memory grant	None	None, unless a Sort is added	Yes, for the hash table
Blocking	No (streaming)	No (streaming)	Build input is consumed first
Cost grows with	Outer rows x inner cost	Sum of both inputs (+ sorts)	Sum of both inputs (+ spills)
Best when	Small outer input and indexed inner side; OLTP lookups	Both inputs large and already ordered (for example by clustered keys)	Large unsorted inputs; analytics; no helpful index
Fails when	Outer input is large, or the inner side scans	A Sort has to be added; many-to-many keys	Estimate is too low: spills; too high: huge grants
Join types	Inner, Left Outer, Left Semi, Left Anti Semi (and cross)	All, including Full Outer and Right variants	All, including Full Outer and Right variants
Typical waits	PAGEIOLATCH_SH (random reads), CXPACKET	SORT-related memory waits, CXPACKET	RESOURCE_SEMAPHORE, IO_COMPLETION (spills), HT* (batch)

*In the graphical plan the **top** input of a join is the outer (Nested Loops) or build (Hash Match) input, and the **bottom** input is the inner or probe input.*

7.2 Nested Loops Join

Algorithm: read the outer input one row at a time. For each outer row, run the inner input looking for matching rows, and emit the matches. In pseudo-code:

```
for each row o in OuterInput:            -- executed once
    for each row i in InnerInput(o):     -- executed once PER outer row (Actual Executions)
        if joinPredicate(o, i): output (o, i)
```

Variants

Variant	Plan clue	Meaning
Dynamic (correlated) loop	Outer References listed	Outer values are pushed into the inner side (for example an index seek key). Most common and fastest form
Static loop	Predicate on the join, no Outer References	Inner input produces the same rows each time and the predicate filters. Can be a disguised cross product; costly with scans
Optimized Nested Loops	Optimized = True	The outer rows are reordered or batched to improve I/O locality when the inner is a seek; helps large lookups from disk
Prefetch	WithOrderedPrefetch / WithUnorderedPrefetch = True	Asynchronous read-ahead for upcoming inner lookups; higher I/O throughput
Apply (CROSS / OUTER APPLY)	Nested Loops (Inner / Left Outer) with Outer References	T-SQL APPLY is implemented as a correlated nested loop
Semi / Anti Semi	Left Semi / Left Anti Semi Join	EXISTS / NOT EXISTS: stops after the first match

Reading a Nested Loops in a plan

- Look at the **Actual Number of Rows** of the outer (top) input. The inner side's **Actual Executions** should equal it. Total inner cost = executions x per-execution cost.
- Compare **Estimated Executions** with Actual. An estimate of 1 execution but 500,000 actual is the classic cause of a plan that is fast for one parameter and a disaster for another.
- Check what the inner side is: an Index Seek (good), a Seek plus Key Lookup (each lookup adds a random read), or a Scan (probably a missing index).
- Rewinds and rebinds on spools show whether the inner result is being reused or rebuilt.

When the optimizer picks it, and when it is wrong

- **Chosen when** the estimated outer row count is small and the inner join column has a suitable index. It needs no memory and returns the first rows quickly.
- **Wrong when** the outer input is much larger than estimated. The cost explodes with outer rows.
- **Fixes:** index the inner join columns (and include needed columns), correct the outer estimate (statistics, parameter sniffing), reduce the outer set earlier, or let the optimizer use hash/merge (avoid forcing LOOP).

```
-- Typical Nested Loops: a few customers, then an indexed seek per customer
SELECT c.CustomerID, o.OrderID, o.OrderDate
FROM dbo.Customers c
JOIN dbo.Orders o ON o.CustomerID = c.CustomerID      -- index on Orders(CustomerID) enables the inner
    seek
WHERE c.Region = 'WEST' AND c.IsVip = 1;               -- few outer rows
```

7.3 Merge Join

Algorithm: both inputs must be sorted on the join keys. The operator keeps a pointer in each input; it advances the pointer on the side with the smaller key, and when keys are equal it outputs the matches. Each input is read **once**, so cost is linear in the sum of the two inputs.

```
a = next(A); b = next(B)
while a and b:
    if a.key < b.key:   a = next(A)
    elif a.key > b.key: b = next(B)
    else:               output(a, b); advance (see one-to-many / many-to-many)
```

- **One-to-many:** at least one input has unique keys. A single forward pass is enough and the operator is very cheap.
- **Many-to-many:** duplicates exist in both inputs. SQL Server must keep a group of equal rows from one input in a **worktable in tempdb** and rewind through it. In the plan *Many to Many* = *True*. If the groups are large this becomes slow; fix with a uniqueness constraint (if correct) or by aggregating one side first.
- **Sorts:** if an input is not already ordered the optimizer must add a **Sort** (memory grant, possible spill). A Merge Join preceded by large Sorts is often worse than a Hash Match. Indexes that deliver the order (for example clustered keys) make Merge Join attractive.
- **Residual predicate:** extra conditions beyond the merge key are evaluated on matching rows.
- **Parallel merge joins** need **order-preserving exchanges** which wait for all inputs and can cause intra-query **parallel deadlocks** (exchange spills, CXPACKET waits). Look for sort warnings in exchanges and consider lower DOP or different indexes.
- **Join types:** Merge Join supports inner, outer (left, right, full), semi, anti-semi and union.

7.4 Hash Match (Hash Join)

Algorithm: pick one input (the **build** input, ideally the smaller one, shown on top), hash the join key of each row and store the rows in an in-memory hash table. Then read the other (**probe**) input, hash each key and look in the matching bucket. Equal keys that also satisfy any residual predicate produce output.

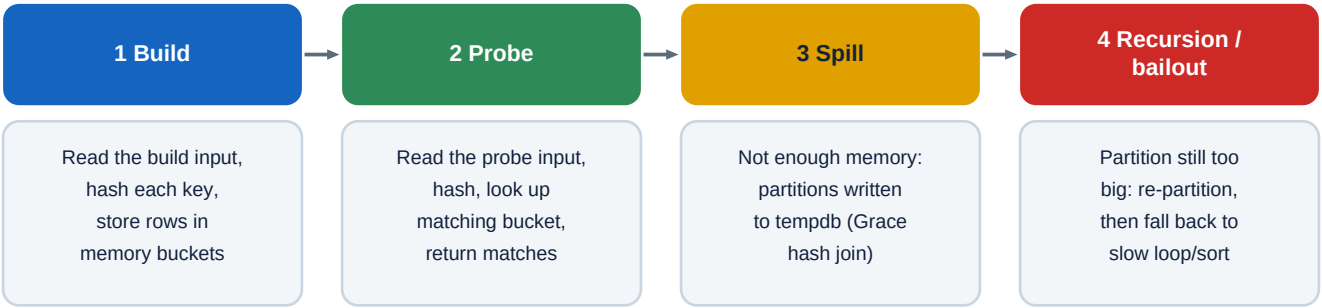


Figure 7.2 - Hash join phases, including spills

Aspect	Detail
Memory	A **memory grant** sized from the estimated build rows and row width. The grant is reserved before the query starts (waits show as RESOURCE_SEMAPHORE)
Spill (Grace hash)	If the build side does not fit, rows are partitioned into tempdb files and joined partition by partition. The plan shows a **Hash Warning** with a spill level
Recursion	If a partition is still too large, it is partitioned again (Hash Warning: hash recursion)
Bailout	If recursion does not help (for example many identical key values), the engine falls back to a slow method (Hash Warning: hash bailout)

Aspect	Detail
Role reversal	If the optimizer's choice of build side is wrong at run time, the roles may swap during spill processing
Bitmap filters	In parallel plans a Bitmap built from the build side filters the probe side early to cut rows before the exchange
Hash teams	Several joins on the same key can share partitioning in one 'hash team', saving memory and repartitioning
Batch mode	Batch mode hash joins process about 900 rows at a time and use specialised waits (HTBUILD, HTREPARTITION and others)
Other uses	The same operator performs hash aggregation, DISTINCT and UNION (Hash Match Union)
Join types	Inner, outer, semi, anti-semi, full outer; equality predicates only

Why hash joins go wrong

- **Build side underestimated:** the grant is too small, so the hash table spills to tempdb (IO_COMPLETION waits, slow queries). Fix the estimate (statistics, sniffing) or reduce row width and row count entering the join.
- **Build side overestimated:** a giant grant reserves memory that others need (RESOURCE_SEMAPHORE waits for other queries). Memory grant feedback and MAX_GRANT_PERCENT help.
- **Wide rows:** the hash table holds all columns the plan carries forward. Select fewer columns or join on narrow key sets and fetch details later.
- **Lack of an index** forces large scans on both inputs; hash is correct but the scans are expensive.

7.5 Adaptive Join

An **Adaptive Join** (SQL Server 2017+, compatibility level 140 and above for batch mode; 150 and above for batch mode on rowstore) starts by reading the build input into a hash-style buffer, counts the rows, and compares with a threshold. Below the threshold it behaves as a **Nested Loops** join; above it, as a **Hash Match**. This avoids committing to one algorithm on an uncertain estimate.

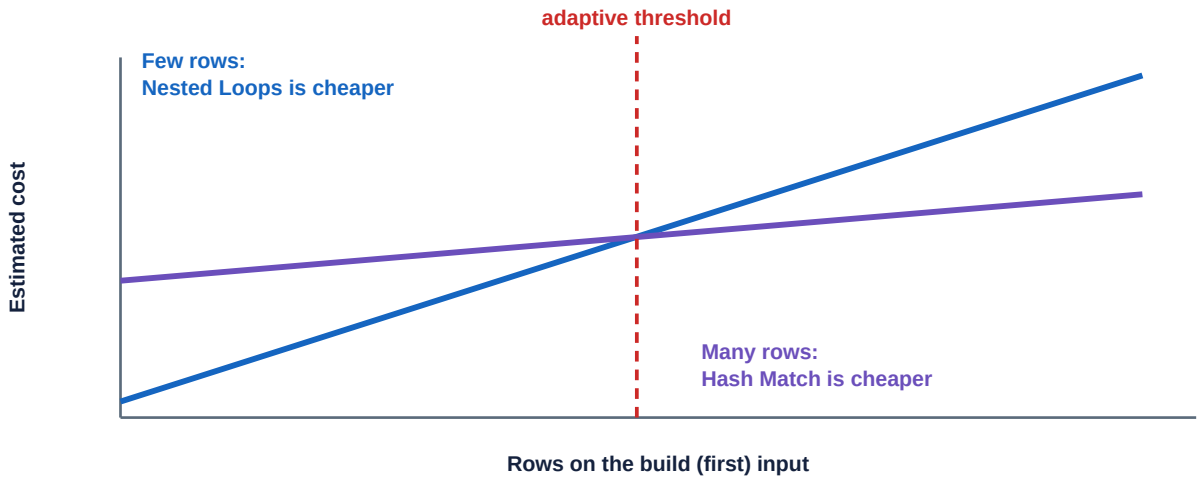


Figure 7.3 - The adaptive threshold where nested loops stops being cheaper than hash

- **Plan properties:** *Adaptive Threshold Rows*, *Estimated Join Type*, *Actual Join Type* (in actual plans). If the actual join type differs from the estimated type, the adaptive join saved you.
- **Requirements:** batch mode execution; supported for inner joins and some other join types depending on version (check the Microsoft documentation for outer and semi join support in your build); the join must be eligible for both algorithms (equality predicate, and an index suitable for nested loops on the inner side).

- **Memory:** the grant is based on the hash option, so an adaptive join can reserve memory even if it ends as a nested loop.
- **Feedback:** memory grant feedback and other IQP features work together with adaptive joins. SQL Server 2025 extends adaptive join scenarios further; check the release notes for your build.

7.6 Logical join types and physical support

Logical join	Meaning	T-SQL source	Nested Loops	Merge	Hash
Inner Join	Rows with matches on both sides	INNER JOIN	Yes	Yes	Yes
Left Outer Join	All rows from the first input plus matches	LEFT JOIN	Yes	Yes	Yes
Right Outer Join	All rows from the second input plus matches	RIGHT JOIN	No (swapped)	Yes	Yes
Full Outer Join	All rows from both inputs	FULL JOIN	No	Yes	Yes
Left Semi Join	First-input rows that have a match (no duplicates)	EXISTS, IN	Yes	Yes	Yes
Right Semi Join	Second-input rows that have a match	EXISTS, IN (swapped)	No	Yes	Yes
Left Anti Semi Join	First-input rows with **no** match	NOT EXISTS, NOT IN, LEFT JOIN ... IS NULL	Yes	Yes	Yes
Right Anti Semi Join	Second-input rows with no match	NOT EXISTS (swapped)	No	Yes	Yes
Cross Join	Every combination	CROSS JOIN, join with no predicate	Yes	No	No

7.7 Semi joins and anti semi joins

- **EXISTS / IN** become **Left Semi Joins**: the inner side stops at the first match, so duplicates in the inner table do not multiply rows.
- **NOT EXISTS** becomes a **Left Anti Semi Join**. It is generally the safest way to write 'rows with no match'.
- **NOT IN** is dangerous: if the subquery can return a NULL, the result is empty (three-valued logic). The plan carries extra Row Count Spool and Assert-like logic to honour this and can be slow. Prefer NOT EXISTS unless you need NOT IN semantics.
- **LEFT JOIN ... WHERE right.key IS NULL** is also recognised as an anti join in many cases.
- Using **DISTINCT or GROUP BY** after an inner join to remove duplicates often hides a missing semi join. Use EXISTS instead.

```
-- Anti join: customers with no orders
SELECT c.CustomerID FROM dbo.Customers c
WHERE NOT EXISTS (SELECT 1 FROM dbo.Orders o WHERE o.CustomerID = c.CustomerID);
```

7.8 Join order, hints and when to force

- The optimizer explores join **orders** (including bushy shapes). Cost estimates determine the order; with many tables it may stop early (optimizer timeout).
- **Query-level hints:** `OPTION (HASH JOIN)`, `OPTION (MERGE JOIN)`, `OPTION (LOOP JOIN)` limit all joins in the statement to that algorithm; `OPTION (FORCE ORDER)` keeps the written join order.
- **Join-level hints:** `INNER HASH JOIN` and similar force that algorithm and also fix the join order of the query. Use only as a diagnostic or a short-term workaround.
- **Better long-term fixes:** correct statistics, add indexes on join keys, adjust data types, use Query Store plan forcing, or apply Query Store hints rather than editing code.
- A forced loop join is a ticking bomb when data grows; a forced hash join can waste memory on small inputs. Always re-check after data changes.

7.9 Join troubleshooting

Symptom in the plan	Likely cause	Action
Nested Loops with millions of inner executions	Outer estimate far too low	Fix statistics or sniffing; consider a plan with hash join; index inner side
Nested Loops whose inner side is a Scan	No index on the inner join column	Create the index; check data types
Merge Join with Sort operators below it	Inputs are not ordered	Add an index in key order, or accept hash
Merge Join Many to Many = True	Non-unique keys both sides	Add uniqueness or pre-aggregate
Hash Match with Hash Warning	Grant too small for the build input	Fix estimates; reduce width; memory grant feedback
Hash Match build input is the larger table	Row estimates reversed	Update statistics; check data skew
Cross product (No join predicate warning)	Missing ON condition	Add the predicate
Implicit conversion on join keys	Different data types	Align types
Join on expressions (for example UPPER(col))	Non-SARGable join	Persisted computed column or fix collation
Same query, different plans	Parameter sniffing, statistics, hints	See chapter 10

8. Parallelism in Plans

A parallel plan splits work across several worker threads and combines the results. Done well it cuts elapsed time; done badly it burns CPU, causes CXPACKET and THREADPOOL waits and slows other queries.

8.1 Exchange operators

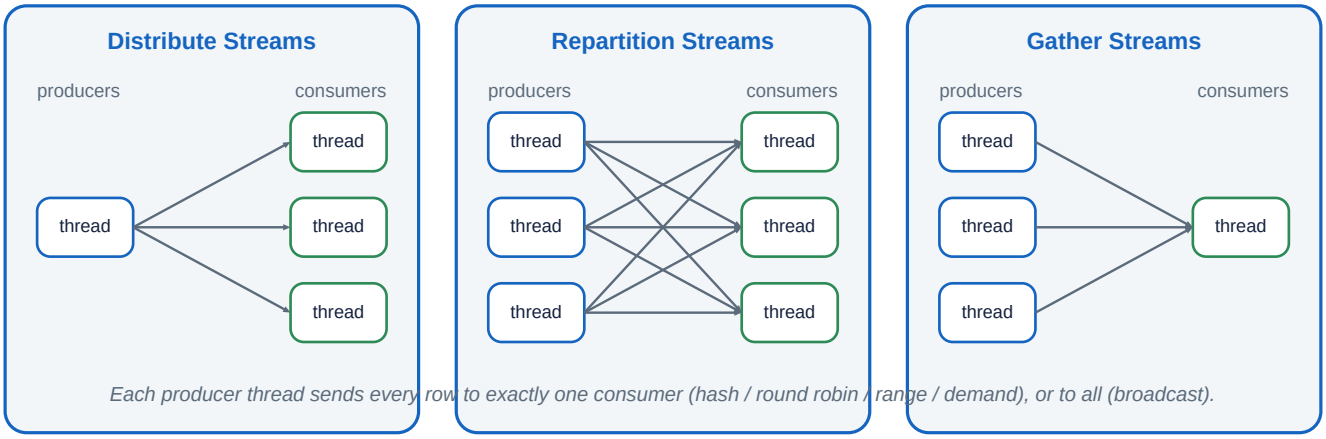


Figure 8.1 - The three kinds of exchange (Parallelism operator)

Partitioning type	How rows are sent	Used for
Hash	Row goes to the thread chosen by a hash of key columns	Parallel joins and aggregates that need matching keys on the same thread
Round Robin	Rows are dealt to threads in turn	Balancing work when no key is needed
Broadcast	Every row goes to every thread	Small input joined to a big input
Demand	Threads pull the next row or page when ready	Parallel scans (page supplier), good load balancing
Range	Rows go to threads by key ranges	Parallel index builds, ordered operations

An exchange has **producer** threads (below it) and **consumer** threads (above it). Between exchanges is a **parallel zone** where several threads run the same operators on different slices of the data. A plan can have serial zones as well, for example above a Gather Streams.

8.2 When the optimizer chooses a parallel plan

- The estimated **serial plan cost** exceeds the **cost threshold for parallelism** (default 5, which is far too low for most modern servers; many teams start at 25 to 50 and test).
- **Max degree of parallelism (MAXDOP)** allows more than one CPU for the query. A server, database-scoped or query-level setting (`OPTION (MAXDOP n)`) caps threads per parallel zone.
- The plan needs **parallel-capable operators**. Several features force a serial plan or a serial zone:

Parallelism inhibitor	Note
Scalar user-defined functions that are not inlined	Forces a serial plan (NonParallelPlanReason: TSQLUserDefinedFunctionsNotParallelizable). Inlining in SQL Server 2019+ removes the problem for eligible functions
Modifications of table variables	INSERT/UPDATE/DELETE into a table variable is serial; reads are parallel

Parallelism inhibitor	Note
System table access, some dynamic management views	Often serial
Multi-statement TVF, some CLR functions	Serial zone or plan
Recursive CTEs, backward scans, certain cursors	Serial zone
Remote queries, OUTPUT INTO and some TOP/row goal shapes	Serial zone
MAXDOP 1, Resource Governor limits, Express edition	Serial

8.3 Reading a parallel plan

- A **yellow circle with two arrows** on an operator icon means it ran in parallel. The Parallelism operators mark zone boundaries.
- On the root, check **Degree of Parallelism** and **Non Parallel Plan Reason**. In actual plans, the thread count per operator is shown (Actual Rows per Thread).
- **Skew:** if one thread processes most rows, the others finish early and wait (CXPACKET / CXCONSUMER). Causes: skewed key distribution after a hash repartition, Round Robin not used, or a serial operator in a parallel zone.
- **Gather Streams** with order-preserving merge waits for all producers; a Gather high in the plan means a large serial region above it.
- Compare **CPU time and elapsed time** from SET STATISTICS TIME: CPU much larger than elapsed shows effective parallelism; CPU about equal to elapsed means the query is effectively serial.

8.4 Parallelism problems and tuning

Problem	Signs	Remedies
Too much parallelism	High CXPACKET, THREADPOOL, CPU saturated by a few queries	Raise cost threshold; lower MAXDOP (commonly at most 8, or per NUMA node); fix the plan so it needs less work; Resource Governor
Skewed threads	One thread with most rows	Check data distribution; update statistics; adjust indexes or use hints sparingly
Exchange spills / parallel deadlock	Sort/exchange warning, sudden slowdowns on merge joins	Reduce DOP; remove order-preserving requirement; change index; test with hash join
Parallel plan slower than serial	More CPU, same or longer duration	Test OPTION (MAXDOP 1); fix estimates; check CPU pressure from other queries
Serial plan where parallel is expected	NonParallelPlanReason set	Remove the inhibitor (inline functions, avoid table variable DML)
Poor DOP for repeated queries	Varies by workload	Use DOP feedback (SQL Server 2022+; on by default in 2025 at the matching compatibility level), or a Query Store hint for MAXDOP

Settings to review on every server
Cost threshold for parallelism, MAXDOP (server and database-scoped), 'optimize for ad hoc workloads', and for SQL Server 2022+ whether DOP feedback is enabled. Revisit after hardware or workload changes.

9. Memory Grants and Spills

Sorts, hash joins and hash aggregates need working memory. The size of that memory is decided **before the query starts**, from the optimizer's estimates. Wrong estimates lead to either spills (too little) or waste and queueing (too much).

9.1 How a memory grant works

- At compile time the optimizer calculates **required memory** (minimum to run) and **additional memory** (to avoid spilling), based on estimated rows and row sizes of memory-consuming operators.
- At execution time the query asks for its grant. If memory is available it is granted immediately; otherwise the query waits in the **resource semaphore queue** (wait type `RESOURCE_SEMAPHORE`).
- The grant is shared across the plan's operators (see Memory Fractions). It is **reserved** for the life of the query, even if only part is used.
- A single query may not take more than a limited share of the available memory (the Resource Governor default is 25% per request). Very large inputs spill instead.
- Queries with small grants use a separate 'small query' semaphore so they are not stuck behind big ones.

Field (plan Memory Grant Info / DMV)	Meaning
Requested / Granted memory	What the plan wanted vs what was given
Max Used Memory	Peak actually used. Granted much larger than used = wasted memory
Grant Wait Time	Time spent waiting in the semaphore queue
Ideal Memory	Memory that would avoid all spills for the estimated rows
Desired / Required memory	Target vs minimum

```
-- Queries running or waiting for memory grants right now
SELECT session_id, request_id, dop, requested_memory_kb, granted_memory_kb, ideal_memory_kb,
       required_memory_kb, used_memory_kb, max_used_memory_kb, wait_time_ms
FROM sys.dm_exec_query_memory_grants ORDER BY requested_memory_kb DESC;
```

9.2 Operators that use grant memory

Operator	Why it needs memory	Spill indicator
Sort, TopN Sort (large N)	Holds rows while sorting	Sort Warning (spill to tempdb)
Hash Match (join, aggregate, union)	Hash table for the build input / groups	Hash Warning with a spill level
Exchange (parallelism) buffers	Buffers rows between threads	Spill in an exchange
Columnstore inserts and index builds	Compress rowgroups and sort	Smaller rowgroups, slower loads
Window Spool, Table Spool	May keep rows in memory or tempdb	tempdb activity

9.3 Spills

When an operator runs out of its granted memory, it writes data to **tempdb** and reads it back. This is a **spill**. Spills turn memory operations into disk operations and are a leading cause of unpredictable query times.

Spill type	What you see	Notes
Sort spill	Warning on Sort: Sort Warning, with granted memory, used memory and writes to tempdb	Multi-pass sort spills are worse
Hash spill (level 1)	Hash Warning: SpillLevel = 1	One partitioning pass to tempdb
Hash recursion (level 2+)	Higher SpillLevel	Partitions still too large; heavy tempdb use
Hash bailout	Hash Warning: bailout	Falls back to a slow method; often caused by many equal keys
Exchange spill	Warning on Parallelism	Often an order-preserving exchange deadlock

9.4 Memory grant feedback and related IQP features

Feature	Version / level	What it does
Batch mode memory grant feedback	SQL Server 2017, compatibility 140	Adjusts the grant after an execution that spilled or wasted memory
Row mode memory grant feedback	SQL Server 2019, compatibility 150	Same for row mode plans
Persisted and percentile feedback	SQL Server 2022, compatibility 160 (needs Query Store)	Stores feedback with the query so it survives cache eviction; percentile-based to handle varying parameters
Feedback stability	Plan property IsMemoryGrantFeedbackAdjusted	YesStable, YesAdjusting, NoFirstExecution, NoAccurateGrant, NoFeedbackDisabled

Feedback is a safety net, not a cure. It cannot fix a query whose memory need changes wildly between executions, and it starts only after a bad execution.

9.5 Finding and fixing grant problems

```
-- Recent spills and big grants from Query Store
SELECT TOP 20 q.query_id, p.plan_id, rs.avg_query_max_used_memory * 8 AS avg_max_used_kb,
       rs.avg_tempdb_space_used * 8 AS avg_tempdb_kb, rs.count_executions
FROM sys.query_store_runtime_stats rs
JOIN sys.query_store_plan p ON p.plan_id = rs.plan_id
JOIN sys.query_store_query q ON q.query_id = p.query_id
ORDER BY rs.avg_tempdb_space_used DESC;

-- Waiting for grants?  (look for RESOURCE_SEMAPHORE)
SELECT wait_type, waiting_tasks_count, wait_time_ms FROM sys.dm_os_wait_stats WHERE wait_type LIKE
    'RESOURCE_SEMAPHORE%';
```

Problem	Fix
Spills because the estimate is too low	Update statistics; fix sniffing/local variables; avoid table variables; reduce columns carried into the sort or hash; MIN_GRANT_PERCENT hint as a stopgap
Excessive grants and RESOURCE_SEMAPHORE queues	Fix the over-estimate; MAX_GRANT_PERCENT hint; Resource Governor workload group limits; memory grant feedback; reduce concurrency of heavy reports

Problem	Fix
Large sorts	Add an index in the ORDER BY / GROUP BY order; TOP with index; avoid sorting wide rows (select only key columns, then join back)
Wide string columns in sorts/hashes	Grants are sized by estimated row width; variable-length columns assume about half the declared length; avoid oversized varchar(8000) or nvarchar(max) in sorts
Hash aggregates on huge group counts	Pre-aggregate, filter earlier, or provide an index to allow Stream Aggregate

10. Common Plan Patterns and How to Fix Them

These are the problems that show up again and again. Each entry says what the plan looks like, why it happens and how to fix it. Test every change and compare reads, CPU and the new plan before and after.

10.1 Key lookups on large row counts

In the plan: an Index Seek followed by a Key Lookup (or RID Lookup) joined by Nested Loops, with tens of thousands of executions. **Cause:** the nonclustered index finds the rows but lacks the other columns. **Fix:** add the needed columns as INCLUDE columns, narrow the SELECT list, or accept lookups when rows are few.

```
-- Before: seek + key lookup for Quantity and UnitPrice
SELECT OrderID, OrderDate, Quantity, UnitPrice FROM dbo.OrderLines WHERE ProductID = 42;
-- After: covering index (no lookup)
CREATE INDEX IX_OrderLines_Product ON dbo.OrderLines (ProductID) INCLUDE (OrderID, OrderDate, Quantity,
    UnitPrice);
```

10.2 Scan instead of seek: non-SARGable predicates

```
-- Bad: function on the column prevents a seek
WHERE YEAR(OrderDate) = 2026                          WHERE LEFT(Name, 3) = 'Abc'
WHERE ISNULL(Status, 0) = 1                           WHERE Name LIKE '%son'
WHERE CONVERT(varchar(10), OrderDate, 120) = '2026-05-01'

-- Good: keep the column bare, move the logic to the value side
WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'
WHERE Name LIKE 'Abc%'
WHERE Status = 1 OR Status IS NULL    -- or store 0 instead of NULL
```

Other fixes: a **computed column with an index** for an unavoidable expression, a **full-text index** for '%word%' searches, or a trigram-style helper table for contains searches.

10.3 Implicit conversions

In the plan: a warning and `CONVERT_IMPLICIT` in a predicate, often with a scan. **Cause:** data type precedence makes SQL Server convert the *column* (the lower-precedence type) to match the parameter. A varchar column compared to an nvarchar parameter is the classic example, common with ORMs that send Unicode strings. **Fix:** make the parameter type match the column (for example `varchar(20)`) in the application, or change the column type.

```
-- Find plans with implicit conversions that affect seek or estimates
SELECT TOP 20 qp.query_plan, qs.execution_count
FROM sys.dm_exec_query_stats qs CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE CAST(qp.query_plan AS nvarchar(max)) LIKE '%PlanAffectingConvert%'
ORDER BY qs.total_worker_time DESC;
```

10.4 Parameter sniffing

In the plan: compiled parameter value differs from the runtime value; the plan is perfect for one value and terrible for another (nested loops with lookups for a popular value, or a scan plan for a rare value). **Cause:** the plan is built for the first parameter set seen and then reused. **It is a feature, not a bug**: plan reuse saves compile CPU. The problem is skewed data.

Option	How	Notes
OPTION (RECOMPILE)	Compile for every execution	Always correct plan; costs compile CPU; good for infrequent or highly variable queries
OPTIMIZE FOR (@p = value) / UNKNOWN	Compile for a typical value or an average	Stable but may be wrong for others
Parameter Sensitive Plan optimization	Database compatibility 160+ (SQL Server 2022): up to three plan variants chosen by a dispatcher	Automatic; look for Dispatcher and QueryVariantID in the plan
Optional Parameter Plan Optimization	Compatibility 170 (SQL Server 2025): separate plans for NULL / NOT NULL optional parameters	For 'WHERE (@p IS NULL OR col = @p)' procedures
Query Store plan forcing or hints	Force a good plan or add OPTION(RECOMPILE) / MAXDOP through sp_query_store_set_hints without changing code	Needs Query Store; hints available from SQL Server 2022
Split logic	Separate procedures or branches for common and rare cases; dynamic SQL for optional filters	Most robust, more code
USE HINT ('DISABLE_PARAMETER_SNIFFING') / PARAMETER_SNIFFING = OFF	Uses density averages for all parameters	Blunt; can hurt queries that benefit from sniffing

```
-- Query Store hint example (SQL Server 2022+): recompile one query without touching code
EXEC sys.sp_query_store_set_hints @query_id = 12345, @query_hints = N'OPTION (RECOMPILE)';
```

10.5 Local variables and the 'kitchen sink' search

A local variable hides its value from the optimizer at compile time, so the estimate comes from density averages. The 'optional parameter' pattern `WHERE (@Name IS NULL OR Name = @Name)` produces one plan that suits no combination. Fix with `OPTION (RECOMPILE)`, **parameterised dynamic SQL** that only includes the active filters, or (SQL Server 2025, compatibility 170) **OPPO**.

```
-- Dynamic SQL with only the filters that are present (still parameterised, plans are cached per shape)
DECLARE @sql nvarchar(max) = N'SELECT OrderID FROM dbo.Orders WHERE 1 = 1';
IF @CustomerID IS NOT NULL SET @sql += N' AND CustomerID = @CustomerID';
IF @From IS NOT NULL       SET @sql += N' AND OrderDate >= @From';
EXEC sys.sp_executesql @sql, N'@CustomerID int, @From date', @CustomerID, @From;
```

10.6 Table variables and multi-statement functions

- **Table variables** have no statistics (estimate 1 row before SQL Server 2019 compatibility 150). A loop join against a table variable holding 100,000 rows is a classic disaster. Use **temp tables** (statistics, indexes) for more than a few hundred rows, or run at compatibility 150+ for deferred compilation.
- **Multi-statement table-valued functions** return a table variable with a fixed estimate and block parallelism. Rewrite as **inline TVFs** (single SELECT), which the optimizer expands into the outer query.
- **Scalar user-defined functions** run per row and block parallelism. SQL Server 2019+ can inline many of them (compatibility 150+); otherwise rewrite as inline TVFs or move the logic into the query.

10.7 Correlated subqueries, spools and row-by-row work

In the plan: an Index Spool or Table Spool on the inner side of a Nested Loops, or Compute Scalar calling a function per row. **Fix:** add the index the spool is simulating, rewrite the subquery as a join or APPLY with a TOP, or pre-aggregate in a derived table.

10.8 Sort spills and ORDER BY problems

- Add an index that returns rows in the ORDER BY order so the Sort disappears. Remember ASC/DESC and column order.
- Sort fewer rows and fewer columns: apply TOP or filters first, sort narrow keys, then join back for wide columns.
- Fix the row estimate so the Sort gets a proper memory grant; watch for sort warnings.

10.9 DISTINCT and GROUP BY that hide duplicates

A plan with an inner join followed by a Hash Match or Sort (Distinct) over many rows suggests the join multiplies rows and DISTINCT cleans up. Use EXISTS (semi join) or aggregate the many-side before joining.

10.10 Paging with large offsets

```
-- OFFSET/FETCH must read and discard all skipped rows
SELECT OrderID FROM dbo.Orders ORDER BY OrderID OFFSET 500000 ROWS FETCH NEXT 50 ROWS ONLY;
-- Keyset (seek) paging: constant cost per page
SELECT TOP (50) OrderID FROM dbo.Orders WHERE OrderID > @LastSeenOrderID ORDER BY OrderID;
```

10.11 Window functions with the default frame

```
-- Default frame is RANGE UNBOUNDED PRECEDING: uses a disk-based Window Spool
SELECT OrderID, SUM(Amount) OVER (ORDER BY OrderDate) AS running_total FROM dbo.Orders;
-- Specify ROWS to get the fast in-memory spool (or batch mode Window Aggregate)
SELECT OrderID, SUM(Amount) OVER (ORDER BY OrderDate, OrderID ROWS BETWEEN UNBOUNDED PRECEDING AND
    CURRENT ROW) FROM dbo.Orders;
```

10.12 Long IN / OR lists

Very long IN lists produce large Constant Scans with Merge Interval and often poor estimates or optimizer timeouts. Load the values into a temp table or table-valued parameter and join.

10.13 Big UPDATE and DELETE statements

- Wide plans (Split, Sort, Collapse) and Eager Spools appear when many rows or many indexes are touched. Log growth and blocking follow.
- **Batch** large changes (for example 5,000 to 50,000 rows per transaction), keep transactions short, and drop unused indexes to lower write cost.
- Ensure foreign key columns are indexed so Assert checks use seeks.

```
WHILE 1 = 1
BEGIN
    DELETE TOP (10000) FROM dbo.AuditLog WHERE CreatedAt < DATEADD(DAY, -90, SYSDATETIME());
    IF @@ROWCOUNT = 0 BREAK;
END
```

10.14 Ad hoc plan cache bloat

Thousands of single-use plans waste memory and compile CPU. Use parameterised queries, enable **optimize for ad hoc workloads**, and consider forced parameterization only after testing. Group by `query_hash` to see how many variants of the same query exist.

11. Version Features and Plan Control

Newer SQL Server versions add features that change plans automatically (Intelligent Query Processing, IQP) and give you more ways to steer them without editing code.

11.1 Intelligent Query Processing by version

Version (compat level)	Feature	Effect and plan clue
2016 (130)	Query Store, Live Query Statistics, batch mode improvements, actual operator time statistics	History of plans; per-operator timing in actual plans
2017 (140)	Adaptive joins (batch), batch mode memory grant feedback, interleaved execution for MSTVFs, automatic plan correction	Adaptive Join operator; IsAdaptive; Interleaved Execution property
2019 (150)	Batch mode on rowstore, row mode memory grant feedback, table variable deferred compilation, scalar UDF inlining, APPROX_COUNT_DISTINCT, last query plan stats, lightweight profiling	Batch execution mode on rowstore scans; correct estimates for table variables; no UDF Compute Scalar after inlining
2022 (160)	Parameter Sensitive Plan optimization, CE feedback, DOP feedback, persisted/percentile memory grant feedback, optimized plan forcing, Query Store hints, Query Store on by default for new databases	Dispatcher + QueryVariantID (PSP); feedback entries in sys.query_store_plan_feedback
2025 (170)	Optional Parameter Plan Optimization, OPTIMIZED_SP_EXECUTESQL, DOP feedback on by default, expanded CE feedback, vector index seek, optimized locking, Query Store on readable secondaries	Multiple plan variants for optional parameters; Vector Index Seek operator

Availability depends on edition, compatibility level and database-scoped settings. Verify with the Microsoft 'Intelligent query processing' documentation for your build.

11.2 Raising compatibility level safely

- Enable **Query Store** and let it capture a baseline at the old compatibility level.
- Raise the compatibility level in a test copy, replay or run the workload, and compare plans and runtime stats in Query Store (Regressed Queries report).
- In production, raise the level, then use **plan forcing** or Query Store hints for any regressions while you fix the cause.
- Use the database-scoped setting `LEGACY_CARDINALITY_ESTIMATION` or the query hint `USE HINT('FORCE_LEGACY_CARDINALITY_ESTIMATION')` for individual queries that dislike the new estimator.

11.3 Plan control tools

Tool	Use	Risk / note
OPTION (RECOMPILE)	Fresh plan per execution using actual values	Compile CPU every time; fine for infrequent queries
OPTIMIZE FOR	Plan for a specified value or average	Becomes wrong when data changes
OPTION (MAXDOP n)	Limit parallelism for one query	Overrides server settings
Table hints: INDEX = , FORCESEEK, FORCESCAN, NOEXPAND	Force an index or access method	Can break when indexes change; use sparingly
Join hints and OPTION (LOOP/HASH/MERGE JOIN, FORCE ORDER)	Force join algorithm or order	Fragile; see chapter 7
USE HINT ('...')	Supported named hints: FORCE_LEGACY_CARDINALITY_ESTIMATION, DISABLE_OPTIMIZER_ROWGOAL, DISABLE_PARAMETER_SNIFFING, QUERY_OPTIMIZER_COMPATIBILITY_LEVEL_n and others	Safer than trace flags
Query Store plan forcing	Force a specific plan for a query without code change	Forcing can fail if the plan becomes invalid (schema change)
Query Store hints (2022+)	Attach hints to a query by id	No code change; visible in sys.query_store_query_hints
Automatic plan correction	Auto-force the last good plan after a regression (FORCE_LAST_GOOD_PLAN)	Verify in sys.dm_db_tuning_recommendations
Plan guides	Older way to attach hints to text you cannot change	Fragile, text-matching; prefer Query Store hints
USE PLAN 'xml'	Force a plan given as XML	Last resort; complex

```
-- Force the best known plan for a regressed query in Query Store
EXEC sys.sp_query_store_force_plan @query_id = 12345, @plan_id = 6789;
-- Remove forcing when no longer needed
EXEC sys.sp_query_store_unforce_plan @query_id = 12345, @plan_id = 6789;
```

12. A Tuning Workflow with Worked Examples

Good tuning is a loop, not a trick. Work the same way every time and change one thing at a time.

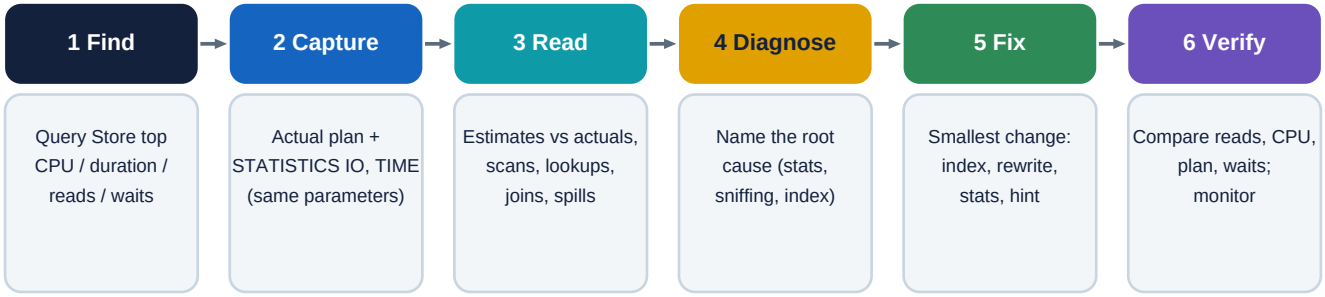


Figure 12.1 - The plan-based tuning loop

Step	Questions to ask
Find	Is this query really a top consumer? By CPU, duration, logical reads, memory or waits? How often does it run? (Frequency x cost)
Capture	Do I have the actual plan with the same parameter values and SET options as the slow case? STATISTICS IO and TIME?
Read	Where do actual rows differ from estimates? Which operator does most reads and time? Any warnings, spills, lookups, scans?
Diagnose	What is the single root cause: statistics, parameter sniffing, missing/poor index, non-SARGable code, schema, memory, parallelism?
Fix	What is the smallest safe change: update statistics, add index, rewrite predicate, hint, Query Store force? Test on a copy.
Verify	Compare logical reads, CPU, elapsed time, plan shape and waits before and after. Check other queries on the same objects. Monitor after release.

12.1 Case 1: fast for one customer, slow for another

Plan evidence	Index Seek + Key Lookup + Nested Loops; compiled CustomerID = 7 (12 rows), runtime CustomerID = 1 (2,100,000 rows); estimated executions 12, actual 2,100,000
Diagnosis	Parameter sniffing on a skewed column
Fix options	Query Store hint OPTION (RECOMPILE); or PSP optimization at compatibility 160; or a covering index so lookups disappear; or separate code path for large customers
Result to expect	Large-customer runs switch to a scan with hash join; reads fall from millions of random lookups to a sequential scan; small customers keep seeks

12.2 Case 2: report spills and takes minutes

Plan evidence	Hash Match with Hash Warning (spill level 2) and Sort Warning; estimated rows 1, actual 4,000,000; the source is a table variable
Diagnosis	Table variable without statistics gave a 1-row estimate, so grants were tiny
Fix	Replace the table variable with a #temp table (and index if needed), or run at compatibility 150+ for deferred compilation; update statistics if staging data changes
Result to expect	Correct estimate, larger grant, no spill; tempdb I/O drops and elapsed time becomes stable

12.3 Case 3: a query that ignores its index

Plan evidence	Clustered Index Scan reading 30M rows, returning 900; CONVERT_IMPLICIT warning on AccountNumber; parameter is nvarchar, column is varchar(20)
Diagnosis	Implicit conversion of the column prevents a seek (data type precedence)
Fix	Change the application parameter type to varchar(20) (or cast the parameter, not the column)
Result to expect	Index Seek, logical reads drop from hundreds of thousands to a few; plan warning disappears

12.4 One-page checklist

Done?	Check
[ ]	Statement facts: compile time, early abort, DOP, memory grant, compiled vs runtime parameters
[ ]	Estimated vs actual rows on every significant operator; note ratios of 10x or more
[ ]	Scans: rows read vs rows returned; non-SARGable predicates; missing index
[ ]	Lookups: executions and whether a covering index is justified
[ ]	Joins: type fits row counts; inner executions; many-to-many; hash warnings
[ ]	Sorts, spools and spills; memory grant waste or shortage
[ ]	Parallelism: skew, serial zones, exchange spills
[ ]	Warnings (implicit conversion, missing statistics, no join predicate)
[ ]	Waits in the plan and Query Store; STATISTICS IO and TIME
[ ]	Single change tested; before/after metrics recorded; rollback plan

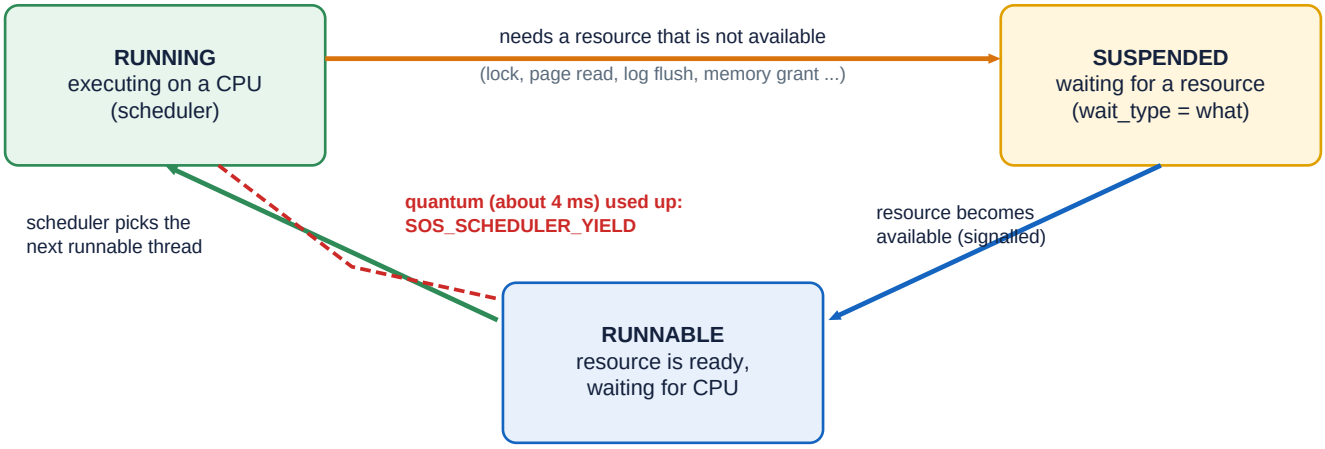
Part II: Wait Types

Part I explained how SQL Server *plans* to run a query. Part II explains where queries *actually spend their time* when they run. Every moment a request is not using the CPU, SQL Server records **why** it is waiting. Those reasons are called **wait types**, and they are the fastest way to find the real bottleneck.

13. Wait Statistics Fundamentals

13.1 What a wait is

SQL Server schedules its own worker threads on each CPU through the **SQLOS scheduler**. A thread is always in one of three states:



wait_time_ms = resource wait (time suspended) + signal wait (time runnable)

Figure 13.1 - The life of a worker thread and where wait time comes from

- **RUNNING** - executing on a CPU. This is productive time; it is not counted as a wait.
- **SUSPENDED** - the thread needs something that is not ready (a lock, a page from disk, a log flush, a memory grant, a client to read results) and waits. The reason is the **wait type**. This time is the **resource wait**.
- **RUNNABLE** - the thing it waited for is ready, but all CPUs are busy, so it stands in the runnable queue. This time is the **signal wait**. High signal wait means CPU pressure.

A thread also moves from RUNNING to RUNNABLE when its 4 ms quantum expires; that is the `SOS_SCHEDULER_YIELD` wait. **Total wait time = resource wait + signal wait**.

13.2 The three families of waits

Family	Meaning	Examples	How to treat
Resource waits	A worker requests something that another worker is using or that is not yet available	Locks, latches, disk and network I/O, memory grants, log flushes	The real performance signals. Investigate the top ones
Queue waits	A background worker is idle, waiting for work to arrive	LAZYWRITER_SLEEP, CHECKPOINT_QUEUE, SLEEP_TASK, XE_TIMER_EVENT	Ignore. They are idle time of system tasks
External waits	A worker waits for something outside the engine: an extended procedure, a linked server, OS calls	OLEDB, PREEMPTIVE_OS_*, MSQL_XP	Investigate the external system, since the worker may be actively running outside code

13.3 Where to read waits

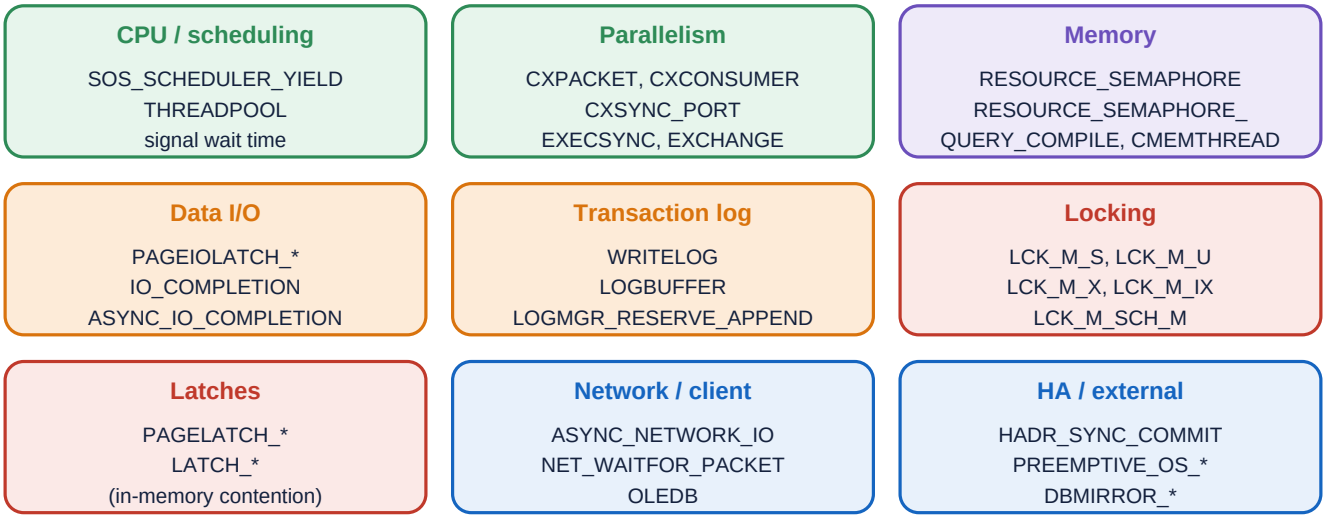


Figure 13.2 - A map from resource to the wait types that signal pressure on it

Source	What it shows	Use it for
sys.dm_os_wait_stats	Cumulative waits for the whole instance since startup or last clear (completed waits only)	Instance-level top waits; compare deltas over time
sys.dm_exec_session_wait_stats	The same, per session	What one connection has waited on
sys.dm_os_waiting_tasks	Tasks waiting right now, with blocking task and resource description	Live diagnosis of blocking and contention (includes parallel worker tasks)
sys.dm_exec_requests	Per request: wait_type, wait_time, last_wait_type, wait_resource, blocking_session_id	Current wait of each running query
Plan XML WaitStats / Query Store waits	Top waits for one query execution, or per query/plan over time (grouped into wait categories)	Link a slow query to its waits
sys.dm_os_latch_stats, sys.dm_os_spinlock_stats	Latch classes and spinlock backoffs	Dig into LATCH_* and CPU-heavy contention

Source	What it shows	Use it for
sys.dm_os_schedulers	runnable_tasks_count, work_queue_count, current_workers_count	CPU queue length and worker shortage
sys.dm_db_wait_stats (Azure SQL Database)	Per-database waits	Cloud database level
Extended Events wait_info, wait_completed	Individual waits with duration and resource	Capture rare long waits
Performance Monitor 'SQLServer:Wait Statistics'	Waits started per second by broad category	Trend charts

The DMV needs VIEW SERVER STATE (VIEW SERVER PERFORMANCE STATE in SQL Server 2022 and later). Counters reset on restart or with DBCC SQLPERF('sys.dm_os_wait_stats', CLEAR). SQL Server has well over a thousand wait types, and many are internal-use only.

13.4 How to read wait statistics properly

- **Use deltas, not totals.** Totals accumulate from startup and hide today's problem under months of history. Take two snapshots and subtract.
- **Rank by wait_time_ms, then check count and average.** One huge wait is different from millions of tiny ones.
- **Remove benign waits** (idle and background) before ranking. The list is in section 14.14.
- **Compute signal wait percentage**: signal_wait_time_ms / wait_time_ms. A sustained value above roughly 20 to 25% suggests CPU pressure.
- **Waits point to a resource, not to a cause.** PAGEIOLATCH says 'reading from disk', not whether storage is slow or queries read too much. Follow up with the matching DMVs and the plan.
- **Take a baseline** of a healthy day. Known-normal waits are the reference for 'abnormal'.

```
-- Top waits since startup with benign waits excluded (see section 14.14 for the full exclusion list)
WITH w AS (
  SELECT wait_type, wait_time_ms / 1000.0 AS wait_s, (wait_time_ms - signal_wait_time_ms) / 1000.0 AS
      resource_s,
         signal_wait_time_ms / 1000.0 AS signal_s, waiting_tasks_count AS wait_count,
         100.0 * wait_time_ms / SUM(wait_time_ms) OVER () AS pct
  FROM sys.dm_os_wait_stats
  WHERE wait_type NOT IN
      ('SLEEP_TASK','LAZYWRITER_SLEEP','CHECKPOINT_QUEUE','XE_TIMER_EVENT','XE_DISPATCHER_WAIT',
        'REQUEST_FOR_DEADLOCK_SEARCH','SQLTRACE_INCREMENTAL_FLUSH_SLEEP','BROKER_TO_FLUSH','BROKER_EVENT
            HANDLER','DIRTY_PAGE_POLL',
        'HADR_FILESTREAM_IOMGR_IOCOMPLETION','LOGMGR_QUEUE','ONDEMAND_TASK_QUEUE','WAITFOR','SP_SERVER_D
            IAGNOSTICS_SLEEP',
        'QDS_PERSIST_TASK_MAIN_LOOP_SLEEP','QDS_ASYNC_QUEUE','FT_IFTS_SCHEDULER_IDLE_WAIT','SLEEP_SYSTEM
            TASK','BROKER_TASK_STOP',
        'BROKER_TRANSMITTER','SQLTRACE_BUFFER_FLUSH','CLR_AUTO_EVENT','CLR_MANUAL_EVENT','DISPATCHER_QUE
            UE_SEMAPHORE'))
)
SELECT TOP 15 wait_type, CAST(wait_s AS decimal(18,1)) AS wait_s, CAST(resource_s AS decimal(18,1)) AS
    resource_s,
       CAST(signal_s AS decimal(18,1)) AS signal_s, wait_count, CAST(pct AS decimal(5,1)) AS pct,
       CAST(wait_s * 1000.0 / NULLIF(wait_count, 0) AS decimal(18,2)) AS avg_ms
FROM w ORDER BY wait_s DESC;
```

```
-- Waits over a 60-second window (delta snapshot)
SELECT wait_type, waiting_tasks_count, wait_time_ms, signal_wait_time_ms INTO #w1 FROM
    sys.dm_os_wait_stats;
WAITFOR DELAY '00:01:00';
SELECT TOP 15 a.wait_type, a.waiting_tasks_count - b.waiting_tasks_count AS waits,
       a.wait_time_ms - b.wait_time_ms AS wait_ms, a.signal_wait_time_ms - b.signal_wait_time_ms AS
           signal_ms
FROM sys.dm_os_wait_stats a JOIN #w1 b ON b.wait_type = a.wait_type
WHERE a.wait_time_ms > b.wait_time_ms ORDER BY wait_ms DESC;
DROP TABLE #w1;
```

```
-- Who is waiting right now, on what, and who blocks them
SELECT wt.session_id, wt.exec_context_id, wt.wait_type, wt.wait_duration_ms, wt.blocking_session_id,
    wt.resource_description
FROM sys.dm_os_waiting_tasks wt WHERE wt.session_id > 50 ORDER BY wt.wait_duration_ms DESC;
```

13.5 Query Store wait categories

Query Store groups waits into categories so you can see which kind of wait hurts a particular query or plan (`sys.query_store_wait_stats`). The categories and the wait types they contain:

Category	Includes (examples)
CPU	SOS_SCHEDULER_YIELD
Worker Thread	THREADPOOL
Lock	LCK_M_%
Latch	LATCH_%
Buffer Latch	PAGELATCH_%
Buffer IO	PAGEIOLATCH_%
Compilation	RESOURCE_SEMAPHORE_QUERY_COMPILE
SQL CLR	CLR%, SQLCLR%
Mirroring	DBMIRROR%
Transaction	XACT%, DTC%, TRAN_MARKLATCH_%, MSQL_XACT_%, TRANSACTION_MUTEX
Idle	SLEEP_%, LAZYWRITER_SLEEP, SQLTRACE_BUFFER_FLUSH, XE_DISPATCHER_WAIT, REQUEST_FOR_DEADLOCK_SEARCH, LOGMGR_QUEUE, ONDEMAND_TASK_QUEUE, CHECKPOINT_QUEUE, XE_TIMER_EVENT
Preemptive	PREEMPTIVE_%
Service Broker	BROKER_% (not BROKER_RECEIVE_WAITFOR)
Tran Log IO	LOGMGR, LOGBUFFER, LOGMGR_RESERVE_APPEND, LOGMGR_FLUSH, LOGMGR_PMM_LOG, CHKPT, WRITELOG
Network IO	ASYNC_NETWORK_IO, NET_WAITFOR_PACKET, PROXY_NETWORK_IO, EXTERNAL_SCRIPT_NETWORK_IOF
Parallelism	CXPACKET, EXCHANGE, CXCONSUMER
Memory	RESOURCE_SEMAPHORE, CMEMTHREAD, CMEMPARTITIONED, EE_PMOLOCK, MEMORY_ALLOCATION_EXT, RESERVED_MEMORY_ALLOCATION_EXT, MEMORY_GRANT_UPDATE
User Wait	WAITFOR, WAIT_FOR_RESULTS, BROKER_RECEIVE_WAITFOR
Tracing	TRACEWRITE, SQLTRACE_LOCK, SQLTRACE_FILE_BUFFER, SQLTRACE_FILE_WRITE_IO_COMPLETION, QUERY_TRACEOUT, TRACE_EVTNOTIF

Category	Includes (examples)
Full Text Search	FT_RESTART_CRAWL, FULLTEXT GATHERER, MSSEARCH, FT_METADATA_MUTEX and other FT_* waits
Other Disk IO	ASYNC_IO_COMPLETION, IO_COMPLETION, BACKUPIO, WRITE_COMPLETION, IO_QUEUE_LIMIT, IO_RETRY
Replication	SE_REPL_%, REPL_%, HADR_% (not HADR_THROTTLE_LOG_RATE_GOVERNOR), PWAIT_HADR_%, REPLICA_WRITES, FCB_REPLICA_WRITE, FCB_REPLICA_READ
Log Rate Governor	LOG_RATE_GOVERNOR, POOL_LOG_RATE_GOVERNOR, HADR_THROTTLE_LOG_RATE_GOVERNOR, INSTANCE_LOG_RATE_GOVERNOR

14. Wait Type Encyclopedia

The encyclopedia groups waits by the resource they point at. **Major waits** get a full card (meaning, causes, confirmation, fix, concern level). The remaining documented and commonly seen waits are in reference tables. Waits that Microsoft documents as 'internal use only' are grouped by family with advice on how to treat them (section 14.15).

> **How complete is this?**
> SQL Server exposes more than a thousand wait types and new ones appear with each version. This chapter covers every wait type that a DBA normally meets, every documented family, and the rules for reading unfamiliar ones. To list what your instance has: `SELECT wait_type FROM sys.dm_os_wait_stats ORDER BY 1;` and compare with the Microsoft documentation for sys.dm_os_wait_stats.

14.1 CPU and scheduling waits

SOS_SCHEDULER_YIELD

Category	CPU / scheduling
What it means	A running thread used up its time slice (about 4 ms) and voluntarily gave the CPU back, going to the runnable queue. It is the normal sign of CPU-bound work, not a resource shortage on its own.
Common causes	CPU-heavy queries (large scans of cached data, hash joins, sorts), too few CPUs for the workload, spinlock contention, power plan or hypervisor limits (CPU ready/steal time).
How to confirm	High count with high signal wait ratio; runnable_tasks_count above 0 on schedulers (sys.dm_os_schedulers); Windows CPU near 100%; top CPU queries in Query Store or dm_exec_query_stats.
How to fix / reduce	Tune the queries that burn CPU (indexes, plans, fewer reads, avoid scalar UDFs); reduce parallelism that oversubscribes; add or upgrade CPUs; set the OS power plan to High Performance; check virtualisation CPU limits.
Concern level	Low on its own; **high when signal waits are a large share of total waits**.

THREADPOOL

Category	CPU / worker threads
What it means	A request is waiting for a free **worker thread**. SQL Server has a limited pool of workers (max worker threads, by default computed from CPU count). When all are busy, new work queues.
Common causes	Blocking chains that pile up many sessions each holding a worker, too many concurrent parallel queries (each uses many workers), very long running queries, connection storms.
How to confirm	THREADPOOL appears in sys.dm_os_wait_stats and in sys.dm_os_waiting_tasks; sys.dm_os_schedulers shows work_queue_count > 0; new connections time out; you may need the DAC to log in.
How to fix / reduce	Fix the root cause: resolve head blockers, reduce DOP and raise cost threshold for parallelism, use connection pooling, fix slow queries. Raising max worker threads rarely helps and costs memory.
Concern level	**Serious.** Any sustained THREADPOOL means the server cannot accept work.

SOS_PHYS_PAGE_CACHE and related SOS_* waits

Category	CPU / memory internals
What it means	Mutexes and synchronisation inside SQLOS memory and scheduling code (for example the lock held while allocating or returning physical pages to Windows).
Common causes	Large memory allocations, lock pages in memory/large pages, memory resizing, NUMA effects. Most SOS_* waits not listed below are internal.
How to confirm	Appear with memory pressure or after changing memory settings; correlate with sys.dm_os_memory_clerks and Windows memory counters.
How to fix / reduce	Usually no action. Investigate only if they dominate: check max server memory, NUMA configuration, and that the OS is not paging.
Concern level	Low unless dominant.

Wait type	What it means	Typical causes	What to do
SOS_WORK_DISPATCHER	Dispatcher worker idle waiting for work	Background	Ignore
SOS_SMALL_PAGE_ALLOC	Allocating small pages from the memory manager	Memory churn	Usually benign; look at memory clerks if high
SOS_VIRTUALMEMORY_LOW	Waiting for virtual memory to become available	Virtual address space pressure (32-bit or very constrained)	Check memory settings and OS limits
SOS_MEMORY_USAGE_ADJUSTMENT	Memory manager adjusting its usage	Memory resize	Informational
SOS_RESERVEDMEMBLOCKLIST	Reserved memory block list synchronisation	Large allocations	Internal; usually benign

14.2 Parallelism waits

Parallel plans produce several related waits. Their meaning has changed across versions, so check yours:

Wait	SQL Server 2016 SP2 / 2017 CU3+	SQL Server 2022+ / Azure SQL
CXPACKET	Waiting to synchronise the exchange iterator and for producers to produce rows	Only waiting for producer threads to produce rows
CXCONSUMER	Consumer thread waiting for producer to send rows (normal)	Same
CXSYNC_PORT	Not present	Opening, closing and synchronising exchange ports between producers and consumers
CXSYNC_CONSUMER	Not present	Waiting to reach a synchronisation point among all consumer threads

CXPACKET (and CXCONSUMER, CXSYNC_PORT, CXSYNC_CONSUMER)

Category	Parallelism
What it means	Threads of a parallel query are waiting for each other at an exchange operator. **CXPACKET only tells you that a parallel query ran**; some of it is always present. CXCONSUMER is a consumer waiting for rows from a producer and is normally harmless.
Common causes	Skewed distribution of rows between threads, one slow thread (waiting on I/O, locks or a serial region), too high a DOP, bad row estimates leading to uneven work, order-preserving exchanges.
How to confirm	In sys.dm_os_waiting_tasks look at the parallel worker tasks of one session (exec_context_id > 0): the thread that is waiting on something **other than CXPACKET** (for example PAGEIOLATCH or LCK_M_*) is the real cause. In the plan check Actual Rows per Thread for skew.
How to fix / reduce	Fix the slow thread's real wait; fix estimates and skew; raise the cost threshold for parallelism and lower MAXDOP; add indexes so less data is processed; use DOP feedback where available.
Concern level	Low to medium: harmful only when it dominates waits and queries are slow.

EXCHANGE and EXECSYNC

Category	Parallelism
What it means	EXCHANGE is synchronisation inside the exchange iterator during parallel queries. EXECSYNC is synchronisation in the query processor in areas other than exchange: bitmaps, LOBs and spool operators.
Common causes	Parallel plans with bitmap filters, large object columns (EXECSYNC is common with LOBs) or spools.
How to confirm	Seen alongside CXPACKET in parallel plans; check plan for Bitmap, Spool and LOB columns.
How to fix / reduce	Treat like CXPACKET: reduce unnecessary parallelism, remove LOB columns from the query, improve estimates.
Concern level	Low.

CXROWSET_SYNC

Category	Parallelism
What it means	Waits during a parallel range scan, where threads coordinate which ranges of a rowset they read.
Common causes	Parallel scans of large tables.
How to confirm	Appears with parallel scan plans.
How to fix / reduce	Usually normal; reduce DOP or scan size if dominant.
Concern level	Low.

14.3 Memory waits

RESOURCE_SEMAPHORE

Category	Memory (query execution memory grants)
What it means	A query asked for a memory grant (for sorts, hashes, exchanges) and the requested memory is not available, so the query waits in the grant queue **before it even starts running**.
Common causes	Over-estimated grants (bad cardinality estimates), many concurrent large queries (reports), too little memory for the workload, a few huge queries reserving memory for long periods.
How to confirm	sys.dm_exec_query_memory_grants shows rows with granted_memory_kb NULL and a wait_time_ms; the plan shows grant wait time; Query Store shows memory waits.
How to fix / reduce	Fix estimates and row widths, memory grant feedback, MAX_GRANT_PERCENT or Resource Governor limits, schedule heavy reports apart, add memory, reduce sorts and hash operations.
Concern level	**High** when queries queue for seconds or more.

RESOURCE_SEMAPHORE_QUERY_COMPILE

Category	Memory (compilation)
What it means	Query compilation is throttled because too many compilations are running at once and compile memory is short. New compiles wait at a 'gateway'.
Common causes	Large numbers of ad hoc, unparameterised queries; plan cache churn after cache clears or restarts; very complex queries; low memory.
How to confirm	Compare compilations/sec and recompilations/sec counters; many distinct query texts with the same query_hash.
How to fix / reduce	Parameterise queries, enable optimize for ad hoc workloads, avoid clearing the cache, simplify queries, forced parameterization where justified.
Concern level	Medium to high in compile storms.

RESOURCE_SEMAPHORE_SMALL_QUERY

Category	Memory
What it means	Waiting for the 'small query' semaphore reserved so that small grants are not blocked by large ones.
Common causes	Very high concurrency or severe memory shortage.
How to confirm	Rare; accompanies RESOURCE_SEMAPHORE.
How to fix / reduce	Resolve overall memory pressure as above.
Concern level	Medium.

CMEMTHREAD

Category	Memory (thread-safe memory objects)
What it means	A task waits for access to a thread-safe memory object that many threads allocate from; contention on a single memory allocator, for example the plan cache or a sort cache.
Common causes	Very high rates of ad hoc compiles or temp object creation, many cores sharing one memory object.
How to confirm	High CMEMTHREAD with high CPU; check sys.dm_os_memory_objects for partitioned vs non-partitioned objects.
How to fix / reduce	Parameterise queries and cache plans; apply current service packs (newer builds partition more memory objects automatically); older versions may use documented trace flags only under support guidance.
Concern level	Medium on large servers with ad hoc workloads.

MEMORY_ALLOCATION_EXT and MEMORY_GRANT_UPDATE

Category	Memory
What it means	MEMORY_ALLOCATION_EXT: allocating memory from the internal pool or the OS. MEMORY_GRANT_UPDATE: internal synchronisation when updating memory grants.
Common causes	Large allocations, memory pressure, many concurrent grant changes.
How to confirm	Appear with memory-hungry queries or rapid grant churn.
How to fix / reduce	Reduce memory demand; update statistics; check max server memory.
Concern level	Low unless dominant.

Wait type	What it means	Typical causes	What to do
LOWFAIL_MEMMGR_QUEUE	Waiting for memory to become available for use	Memory exhaustion	Check memory settings, leaks and OS pressure
RESERVED_MEMORY_ALLOCATION_EXT	Allocation of reserved memory	Large reservations	Usually internal
EE_PMOLOCK	Synchronisation of certain memory allocations during statement execution	Memory-heavy executions	Usually benign
CMEMPARTITIONED	Contention on partitioned thread-safe memory objects	As CMEMTHREAD	As CMEMTHREAD
RESOURCE_SEMAPHORE_MUTEX	Mutex around the memory grant resource semaphore	Heavy grant traffic	Reduce grant churn
BUFFERPOOL_SCAN	Waiting for a buffer pool scan (SQL Server 2022+), for example when a task triggers a scan of a very large buffer pool	Operations that scan the buffer pool on large-memory servers	Review operations that trigger buffer pool scans; apply updates
SOS_MEMORY_USAGE_ADJUSTMENT	Internal memory usage adjustment	Memory resize events	Informational

14.4 Data file and disk I/O waits

PAGEIOLATCH_SH (and _EX, _UP, _KP, _DT)

Category	Disk I/O (data pages)
What it means	A task needs a data page that is not in the buffer pool and waits for it to be **read from disk** (SH = shared, the common read case). EX/UP are for buffers being written or modified while in I/O; KP/DT are keep and destroy modes. The suffix is the latch mode.
Common causes	Slow storage, queries that read too much (scans, missing indexes, key lookups), buffer pool too small for the working set (low page life expectancy), cold cache after restart, storage contention from other systems or backups, mis-sized HBA/queue depth, antivirus on data files.
How to confirm	sys.dm_io_virtual_file_stats: read latency per file (stall_read_ms / num_of_reads). Windows 'Avg. Disk sec/Read'. SQLServer:Buffer Manager Page life expectancy and page reads/sec. STATISTICS IO showing physical reads. Query Store 'Buffer IO' wait category.
How to fix / reduce	First reduce reads: indexes, better predicates, fix scans/lookups. Then add memory so more data stays cached. Then improve storage latency (faster tier, spread files, correct multipath and queue depth). Move tempdb and logs off shared slow paths.
Concern level	**High** when average read latency exceeds roughly 10 to 20 ms for OLTP data files, or when PAGEIOLATCH dominates the wait profile.

```
-- Average I/O latency per database file since startup
SELECT DB_NAME(vfs.database_id) AS db, mf.physical_name, mf.type_desc,
       vfs.num_of_reads, vfs.io_stall_read_ms / NULLIF(vfs.num_of_reads, 0) AS avg_read_ms,
       vfs.num_of_writes, vfs.io_stall_write_ms / NULLIF(vfs.num_of_writes, 0) AS avg_write_ms
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
JOIN sys.master_files mf ON mf.database_id = vfs.database_id AND mf.file_id = vfs.file_id
ORDER BY vfs.io_stall DESC;
```

Latch mode	Suffix	Meaning in an I/O wait
Shared	PAGEIOLATCH_SH	Waiting to read a page from disk into memory (most common)
Exclusive	PAGEIOLATCH_EX	Waiting while a buffer is being written to disk
Update	PAGEIOLATCH_UP	Waiting on a buffer in I/O in update mode (for example allocation pages)
Keep	PAGEIOLATCH_KP	Keep latch on a buffer in I/O so it cannot be destroyed
Destroy	PAGEIOLATCH_DT	Waiting to destroy a buffer in I/O
Null	PAGEIOLATCH_NL	Not used

IO_COMPLETION

Category	Disk I/O (non-data pages)
What it means	Waiting for **synchronous I/O that is not a data page read**: sorts and hash joins spilling to tempdb, reading the transaction log during recovery or rollback, some index operations. Data page I/O shows as PAGEIOLATCH instead.
Common causes	Spills to tempdb (large sorts or hash joins), slow tempdb or log storage, rollback of large transactions.
How to confirm	High IO_COMPLETION with sort/hash warnings in plans; tempdb file latency from sys.dm_io_virtual_file_stats.
How to fix / reduce	Remove the spills (estimates, memory grants), move tempdb to fast storage, add tempdb files, reduce rollback volume.
Concern level	Medium; high when it mirrors tempdb latency.

ASYNC_IO_COMPLETION

Category	Disk I/O (asynchronous non-data)
What it means	Waiting for asynchronous non-data I/Os, for example backup and restore operations, database mirroring or log shipping file work, some bulk import steps.
Common causes	Backups and restores running, large file copies, bulk operations.
How to confirm	Occurs during backups; check sys.dm_exec_requests for BACKUP or RESTORE commands.
How to fix / reduce	Normal during backups. If slow, tune backup destination throughput, use compression and striping.
Concern level	Low during maintenance windows.

WRITE_COMPLETION and IO_RETRY

Category	Disk I/O
What it means	WRITE_COMPLETION: a write operation in progress for internal writes such as checkpoint-related file writes. IO_RETRY: an I/O such as a read or write failed because of insufficient resources and is retried.
Common causes	Storage saturation, resource shortage in the I/O path, flaky storage drivers.
How to confirm	Look for I/O error messages in the SQL error log (such as 'taking longer than 15 seconds') and Windows event log.
How to fix / reduce	Investigate storage health and drivers. IO_RETRY repeating is a hardware or driver warning.
Concern level	WRITE_COMPLETION low; IO_RETRY **high** if persistent.

Wait type	What it means	Typical causes	What to do
DISKIO_SUSPEND	A task waits to access a file while an external (snapshot/VSS) backup is active	Third-party or VSS backups taking long	Check the backup product; count over about 5 per user process indicates a slow backup
IMPPROV_IOWAIT	Waiting for a bulk-load I/O to finish	Bulk load from slow source/files	Tune the bulk load path and storage
BACKUPIO, BACKUPBUFFER	A backup task waits for data or for a buffer to write it to; typical only when waiting for a tape mount	Slow backup destination	Faster target, more buffers (BUFFERCOUNT), striped files, compression
BACKUPTHREAD	Waiting for a backup task to finish; can be minutes to hours	Long backups	Normal while a backup runs; investigate duration separately
BACKUP, BACKUP_OPERATOR	A task is blocked as part of backup processing; or waiting for a tape mount	Backup in progress, tape devices	Check tape status via sys.dm_io_backup_tapes if used
PREEMPTIVE_OS_WRITEFILEGATHER	Zero-initialising file space for file growth (log files always; data files when instant file initialisation is off)	Autogrowth events	Enable instant file initialisation for data files; pre-size files; sensible growth increments

Wait type	What it means	Typical causes	What to do
FCB_REPLICA_READ / FCB_REPLICA_WRITE	Synchronising reads and writes of a snapshot or DBCC temporary snapshot sparse file	DBCC CHECKDB with snapshots, database snapshots	Expected during DBCC on busy systems
IO_QUEUE_LIMIT	I/O queue limit reached (internal)	Heavy asynchronous I/O	Internal
DIRTY_PAGE_POLL, DIRTY_PAGE_SYNC, DIRTY_PAGE_TABLE_LOCK	Internal dirty page tracking	Background	Usually ignore
CHKPT, CHECKPOINT_QUEUE	Checkpoint thread start-up / idle waiting for the next checkpoint request	Background	Ignore
LAZYWRITER_SLEEP	Lazy writer suspended (idle)	Background	Ignore

14.5 Transaction log waits

WRITELOG

Category	Transaction log I/O
What it means	A session is waiting for its **log records to be flushed (hardened) to the log file**, usually at COMMIT. Every durable transaction must wait for this write before it is acknowledged.
Common causes	Slow log storage latency; very many tiny commits (autocommit loops); a log disk shared with other busy files; synchronous availability group replicas (also HADR_SYNC_COMMIT); too many VLFs; antivirus; virtualization storage layers.
How to confirm	sys.dm_io_virtual_file_stats on the log file: average write latency (aim for about 1 to 2 ms, investigate above 5 ms). Perfmon 'Avg. Disk sec/Write' on the log volume. Transaction counters: Log Flush Wait Time, Log Flushes/sec.
How to fix / reduce	Put the log on fast, low-latency storage (SSD/NVMe, write cache); batch work into larger transactions or use explicit transactions around loops; reduce log volume (minimal logging, set-based changes); consider delayed durability only if some data loss is acceptable; keep VLF count reasonable.
Concern level	**High** for OLTP: every commit waits on it.

LOGBUFFER

Category	Transaction log
What it means	A task waits for space in the in-memory **log buffer** to store a log record. The log writer cannot flush fast enough to free buffer space.
Common causes	Log device cannot keep up with log generation: heavy bulk/ETL, huge batch updates, slow log disk.
How to confirm	Consistently high LOGBUFFER together with WRITELOG and log file write latency.
How to fix / reduce	Same fixes as WRITELOG; reduce log generation rate; faster log storage.
Concern level	Medium to high when sustained.

LOGMGR_RESERVE_APPEND

Category	Transaction log (space)
What it means	A task is waiting to see whether **log truncation frees space** so it can write a new log record. The log file is full or at its size limit and cannot grow.
Common causes	Log not truncating (log_reuse_wait_desc shows why: ACTIVE_TRANSACTION, LOG_BACKUP, REPLICATION, AVAILABILITY_REPLICA, etc.), log file at max size or disk full.
How to confirm	SELECT name, log_reuse_wait_desc FROM sys.databases; sys.dm_db_log_space_usage; error 9002 in the log.
How to fix / reduce	Fix the reason log reuse is blocked (take log backups, resolve long transactions, fix AG/replication lag); add log space; right-size the log file.
Concern level	**Critical**: writes are stalled until space is freed.

```
SELECT name, recovery_model_desc, log_reuse_wait_desc FROM sys.databases;          -- why can't the log
    be reused?
SELECT * FROM sys.dm_db_log_space_usage;                                           -- current database
SELECT TOP 5 s.session_id, t.transaction_begin_time, DATEDIFF(MINUTE, t.transaction_begin_time,
    GETDATE()) AS minutes_open
FROM sys.dm_tran_active_transactions t JOIN sys.dm_tran_session_transactions s ON s.transaction_id =
    t.transaction_id
ORDER BY t.transaction_begin_time;                                                 -- oldest open
    transactions
```

Wait type	What it means	Typical causes	What to do
LOGMGR	Waiting for outstanding log I/Os to finish before shutting the log down when closing a database	Closing or detaching a database	Normal at shutdown/close
LOGMGR_FLUSH	Internal log manager flush	Log activity	Informational
LOGMGR_QUEUE	Log writer task waiting for log flush requests (idle)	Background	Ignore
LOGMGR_PMM_LOG	Internal log manager	Log activity	Internal
LOG_RATE_GOVERNOR	Throttled by the log rate governor (cloud service tiers)	Log generation above the tier limit	Scale the tier, reduce log volume, batch smaller
POOL_LOG_RATE_GOVERNOR	Pool-level log rate throttle (elastic pools)	Total pool log rate exceeded	Scale the pool or reduce load
INSTANCE_LOG_RATE_GOVERNOR	Instance-level log rate throttle (managed instance)	Instance log rate limit	Scale or reduce log volume
HADR_THROTTLE_LOG_RATE_GOVERNOR	Log rate throttled because a replica or geo-secondary cannot keep up	Slow secondary	Fix replica lag; match secondary size
HADR_THROTTLE_LOG_RATE_MISMATCHED_SLO	Geo-replication secondary has lower compute size than primary (Azure SQL Database)	Mismatched service objectives	Scale the secondary to match
LOG_POOL_SCAN, LOGPOOL_*	Internal log pool structures used by AG, CDC and replication scans	Log readers	Internal; check consumers if high
LOGCAPTURE_LOGPOOLTRUNCPOINT	Internal log capture synchronisation	Replication/AG	Internal
TRACEWRITE	Waiting for a trace rowset provider to consume events	Profiler/trace consumers	Stop slow traces; use Extended Events

14.6 Lock waits (LCK_M_*)

A **lock wait** means one session is blocked by another that holds an incompatible lock on the same resource. The wait type name encodes the **mode requested**: `LCK_M_` followed by the mode. Lock waits are about **concurrency design** (transactions, indexing, isolation), not hardware.

LCK_M_S / LCK_M_U / LCK_M_X (and IS, IU, IX, SIU, SIX, UIX, BU)

Category	Locking
What it means	S = waiting for a shared (read) lock; U = update lock (read with intent to modify); X = exclusive (write) lock; IS/IX/IU/SIX/SIU/UIX = intent combinations at higher levels (table/page); BU = bulk update. The session cannot proceed until the blocker commits or rolls back.
Common causes	Long-running transactions; updates scanning many rows because of missing indexes; readers blocked by writers in READ COMMITTED (locking) mode; lock escalation to table level; application holding transactions open while waiting for users; large batch operations during business hours.
How to confirm	sys.dm_os_waiting_tasks (blocking_session_id, resource_description); sys.dm_tran_locks; the head blocker is the session blocking others but not blocked itself; blocked process report; Query Store 'Lock' wait category.
How to fix / reduce	Shorten transactions; add indexes so updates touch fewer rows; enable READ_COMMITTED_SNAPSHOT or use SNAPSHOT isolation so readers do not block writers; batch large modifications; avoid user interaction inside transactions; access objects in a consistent order; review lock escalation.
Concern level	**High** when waits are long or blockers are stuck.

LCK_M_SCH_S and LCK_M_SCH_M

Category	Locking (schema)
What it means	SCH_S (schema stability) is taken by every query to prevent the table from changing; SCH_M (schema modification) is taken by DDL such as ALTER TABLE, index rebuilds and some online operations. They are incompatible: DDL waits for all queries to finish, and queued DDL then blocks **new** queries.
Common causes	DDL (ALTER TABLE, CREATE/ALTER INDEX, statistics updates in some cases) run during busy periods; long-running queries holding SCH_S; a pending SCH_M causing a pile-up of LCK_M_SCH_S waits behind it.
How to confirm	A session waiting on LCK_M_SCH_M with a long query ahead of it; many sessions in LCK_M_SCH_S queue behind the DDL.
How to fix / reduce	Run DDL in quiet windows; use online index operations with WAIT_AT_LOW_PRIORITY and ABORT_BLOCKERS/SELF; shorten long queries; avoid DDL in application code at peak.
Concern level	**High** because one DDL can stall a whole workload.

LCK_M_RS_*, LCK_M_RIn_*, LCK_M_RX_* (key-range locks)

Category	Locking (range)
What it means	Range locks protect gaps between index keys. They are used under SERIALIZABLE isolation (or HOLDLOCK) to prevent phantom rows. RangeS-S (RS_S), RangeS-U (RS_U), RangeI-N/S/U/X (RIn_NL, RIn_S, RIn_U, RIn_X: insert range) and RangeX-S/U/X (RX_S, RX_U, RX_X).
Common causes	SERIALIZABLE isolation level or HOLDLOCK/SERIALIZABLE table hints; foreign key checks; some transactional replication or distributed transactions.
How to confirm	Waits whose resource_description shows KEY locks and transaction isolation level 4 (serializable) for the blocking session.
How to fix / reduce	Use a lower isolation level or optimistic isolation when phantoms are acceptable; avoid HOLDLOCK; ensure supporting indexes so ranges are narrow.
Concern level	Medium to high where SERIALIZABLE is used.

..._LOW_PRIORITY and ..._ABORT_BLOCKERS lock waits

Category	Locking (online DDL options)
What it means	Variants such as LCK_M_X_LOW_PRIORITY or LCK_M_SCH_M_ABORT_BLOCKERS appear when ALTER TABLE or ALTER INDEX uses WAIT_AT_LOW_PRIORITY (SQL Server 2014+). The operation waits politely or kills blockers after a timeout.
Common causes	Online index rebuild/ALTER TABLE SWITCH using the low priority wait option.
How to confirm	Seen only when that option is used.
How to fix / reduce	Tune MAX_DURATION and ABORT_AFTER_WAIT (NONE, SELF, BLOCKERS) per your tolerance.
Concern level	Informational.

LCK_M_S_XACT, LCK_M_S_XACT_READ, LCK_M_S_XACT_MODIFY (optimized locking)

Category	Locking (SQL Server 2025 / Azure)
What it means	With **optimized locking** enabled, writers hold a transaction ID (TID) lock instead of many row/page locks. These waits show a task waiting for a shared lock on a transaction resource, with the intent to read, to modify, or undetermined.
Common causes	Contention with an in-flight modifying transaction when optimized locking is on.
How to confirm	Seen only on versions and databases with optimized locking.
How to fix / reduce	Same principles: shorter transactions, indexing, RCSI (required for the full benefit of optimized locking).
Concern level	As for other lock waits.

Lock modes at a glance

Mode	Wait suffix	Meaning	Typical source
Shared (S)	LCK_M_S	Read; many allowed; blocks writers	SELECT under READ COMMITTED (locking)
Update (U)	LCK_M_U	Read with intent to modify; prevents conversion deadlocks	UPDATE/DELETE locating rows
Exclusive (X)	LCK_M_X	Write; blocks everything	INSERT/UPDATE/DELETE
Intent Shared / Exclusive / Update (IS, IX, IU)	LCK_M_IS, LCK_M_IX, LCK_M_IU	Declares intent to lock lower levels (rows/pages)	Table/page level when rows are locked
Shared with Intent Exclusive (SIX), Shared Intent Update (SIU), Update Intent Exclusive (UIX)	LCK_M_SIX, LCK_M_SIU, LCK_M_UIX	Combined intent modes	Scans that also modify rows
Bulk Update (BU)	LCK_M_BU	Bulk load	BULK INSERT with TABLOCK
Schema Stability / Modification	LCK_M_SCH_S, LCK_M_SCH_M	Protect object definitions	All queries / DDL
Key-range modes	LCK_M_RS_*, RIn_*, RX_*	Protect gaps between keys	SERIALIZABLE

Requested \ held	IS	IX	S	U	X
IS	Yes	Yes	Yes	Yes	No
IX	Yes	Yes	No	No	No
S	Yes	No	Yes	Yes	No
U	Yes	No	Yes	No	No
X	No	No	No	No	No

Lock compatibility for the most common modes: 'Yes' means the requested lock can be granted while the other lock is held.

Diagnosing blocking

```
-- Blocking chains right now: who blocks whom, on what, running what
SELECT r.session_id AS blocked, r.blocking_session_id AS blocker, r.wait_type, r.wait_time / 1000.0 AS
    wait_s, r.wait_resource,
       DB_NAME(r.database_id) AS db, SUBSTRING(t.text, 1, 200) AS blocked_sql
FROM sys.dm_exec_requests r CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id <> 0 ORDER BY r.wait_time DESC;

-- Head blockers: sessions that block others but are not blocked
SELECT DISTINCT r.blocking_session_id AS head_blocker
FROM sys.dm_exec_requests r
WHERE r.blocking_session_id <> 0 AND r.blocking_session_id NOT IN (SELECT session_id FROM
    sys.dm_exec_requests WHERE blocking_session_id <> 0);

-- Locks held and requested
SELECT request_session_id, resource_type, resource_description, request_mode, request_status
FROM sys.dm_tran_locks WHERE request_status = 'WAIT' OR request_session_id = 57;
```

Remedy	When it helps
READ_COMMITTED_SNAPSHOT (RCSI) / SNAPSHOT isolation	Readers blocking on writers: readers read the last committed version from the version store (tempdb or PVS)

Remedy	When it helps
Index the filter and join columns of UPDATE/DELETE	Fewer rows scanned and locked; fewer escalations
Shorter, smaller transactions	Reduces how long locks are held
Consistent object access order	Avoids deadlocks
Control lock escalation (table option, batching under about 5,000 locks)	Table locks caused by escalation
Capture deadlocks with the system_health Extended Events session	Deadlocks (LCK_M_* waits ending in error 1205)

Wait type	What it means	Typical causes	What to do
DEADLOCK_ENUM_MUTEX	Deadlock monitor and sys.dm_os_waiting_tasks avoid running multiple deadlock searches at once	Queries on dm_os_waiting_tasks	Usually benign
DEADLOCK_TASK_SEARCH	Large waits mean queries on sys.dm_os_waiting_tasks block the deadlock monitor	Monitoring queries running constantly	Reduce frequency of such monitoring
REQUEST_FOR_DEADLOCK_SEARCH	Deadlock monitor sleeping between searches	Background	Ignore
TRAN_MARKLATCH_*	Waiting on a latch for a transaction mark	Marked transactions (log marks)	Rare
PRINT_ROLLBACK_PROGRESS	Waiting for user processes to end after ALTER DATABASE ... WITH ROLLBACK	Terminating sessions	Normal

14.7 Latch waits

Latches are short, lightweight internal locks that protect in-memory structures (buffer pages, allocation maps, B-tree roots). They are held for microseconds and are not under your transactional control. When many threads hit the same structure, latch waits appear. **PAGEIOLATCH** waits are about disk reads (section 14.4); **PAGELATCH** and **LATCH** waits are about **in-memory contention**.

PAGELATCH_EX / PAGELATCH_SH / PAGELATCH_UP (and _KP, _DT)

Category	Latch (buffer, in memory)
What it means	A task waits for a latch on a buffer page that is **already in memory** and not in I/O. EX = exclusive (writer), SH = shared (reader), UP = update (typically allocation pages such as PFS, GAM, SGAM).
Common causes	**tempdb allocation contention** (PFS/GAM/SGAM pages 2:1:1, 2:1:2, 2:1:3) from heavy temp table creation; **last-page insert hotspot** on an ever-increasing clustered key (identity, datetime); queue tables with random inserts; hot small tables or index root pages.
How to confirm	sys.dm_os_waiting_tasks resource_description in the form database:file:page. In tempdb, pages 1, 2, 3 or multiples of 8088 (PFS) / 511232 (GAM/SGAM) confirm allocation contention. For user databases, find the object with sys.dm_db_page_info.
How to fix / reduce	tempdb: several equally sized data files (commonly up to 8 or the number of cores), current versions use proportional fill and eliminate mixed extents by default, memory-optimized tempdb metadata in SQL Server 2019+. Last-page insert: OPTIMIZE_FOR_SEQUENTIAL_KEY (2019+), hash/partition the clustered key, change key design, batch inserts. Cache and reuse temp tables; reduce temp object churn.
Concern level	Medium to **high** on busy OLTP systems when PAGELATCH dominates and CPU is not saturated.

```
-- Which pages are latched right now?
SELECT wt.session_id, wt.wait_type, wt.wait_duration_ms, wt.resource_description
FROM sys.dm_os_waiting_tasks wt WHERE wt.wait_type LIKE 'PAGELATCH%';
-- Identify the object for a page (SQL Server 2019+)
SELECT * FROM sys.dm_db_page_info(2, 1, 1, 'LIMITED');   -- database_id, file_id, page_id
```

LATCH_EX / LATCH_SH / LATCH_UP / LATCH_KP / LATCH_DT

Category	Latch (non-buffer)
What it means	A task waits for a latch on an internal structure that is **not a buffer page or transaction mark**: B-tree virtual roots, dataset parents for parallel scans, file control blocks, log manager structures and many others. The latch class is the key to understanding it.
Common causes	Specific contention points: ACCESS_METHODS_HOBT_VIRTUAL_ROOT (B-tree root splits), ACCESS_METHODS_DATASET_PARENT (parallel scans), FGCB_ADD_REMOVE (file growth), LOG_MANAGER, DBCC_* classes during DBCC, and others.
How to confirm	sys.dm_os_latch_stats lists latch classes with waiting counts and times. SELECT latch_class, waiting_requests_count, wait_time_ms FROM sys.dm_os_latch_stats ORDER BY wait_time_ms DESC.
How to fix / reduce	Depends on the class: reduce parallel scans, avoid frequent file growth (pre-size), fix hot-page inserts, apply updates, review index design. If unexplained, collect latch stats over time and involve support.
Concern level	Medium; high only when a single class dominates.

LATCH_NL

Category	Latch
What it means	Null latch mode. Documented for information only and not used in practice.
Common causes	-
How to confirm	-
How to fix / reduce	-
Concern level	Ignore.

Spinlocks are an even lighter mechanism that burns CPU instead of waiting. They do not appear as waits but show up as high CPU with SOS_SCHEDULER_YIELD; use `sys.dm_os_spinlock_stats` to look at them.

14.8 Network, client and linked-server waits

ASYNC_NETWORK_IO

Category	Network / client
What it means	SQL Server has results ready to send but is **blocked waiting for the client application to consume them** (acknowledge the data already sent). The server is not slow; the client or network is.
Common causes	Application reading results row by row slowly, writing to a file or UI while reading, waiting for user input mid-result, client-side filtering of huge result sets, overloaded client machine (CPU, memory), network delays (driver, firewall, misconfigured routers), SSMS displaying large grids.
How to confirm	Request status shows ASYNC_NETWORK_IO while the session is active; the client process is busy; result sets are large (check reads and rows returned).
How to fix / reduce	Return fewer rows (filter and aggregate on the server); read results promptly with fast data readers; avoid per-row work inside the read loop; fix client resources; check network adapters, drivers and firewalls; use MARS carefully.
Concern level	Low for the server, but it keeps locks and memory grants open if a transaction or grant is held while waiting. **Medium** when queries hold locks.

OLEDB

Category	External (OLE DB / linked servers)
What it means	SQL Server is calling an OLE DB or OLE DB Driver provider (for example a **linked server query**, OPENROWSET, OPENQUERY). The wait measures the duration of the provider call, not synchronisation.
Common causes	Slow or distant linked servers, large remote result sets, remote queries that cannot push down predicates, DMV-style monitoring tools using OLE DB.
How to confirm	Plans contain Remote Query/Remote Scan operators; sessions show OLEDB and the linked server name in the SQL text.
How to fix / reduce	Pull less data (filter remotely with OPENQUERY), copy remote data locally, index remote tables, review remote server health, avoid joins between large local and remote tables.
Concern level	Medium; depends on remote system.

NET_WAITFOR_PACKET

Category	Network
What it means	A connection is waiting for a network packet during a network read: the client has not yet sent the next request or the rest of a batch.
Common causes	Idle-ish connections between requests, slow clients, network latency.
How to confirm	Seen in session wait stats for connections that do not send data promptly.
How to fix / reduce	Usually no action. High on application/network issues: investigate latency and client behaviour.
Concern level	Low.

Wait type	What it means	Typical causes	What to do
MSQL_DQ	A task is waiting for a distributed query operation to finish (used to detect MARS deadlocks)	Distributed queries	Investigate the remote query; see OLEDB

Wait type	What it means	Typical causes	What to do
MSQL_XP	Waiting for an extended stored procedure to end	xp_cmdshell, third-party extended procedures	Examine the external program; avoid xp_cmdshell in tight loops
PROXY_NETWORK_IO, EXTERNAL_SCRIPT_NETWORK_IOF	Network I/O to a proxy or external script service	Linked resources, R/Python external scripts	Check the external service and its network
DBMIRROR_SEND	Waiting for a network communication backlog to clear before sending mirroring messages	Communication layer overloaded	Network throughput to the partner
PREEMPTIVE_OS_WINSOCKOPS, PREEMPTIVE_OS_GETADDRINFO	Windows socket operations and name resolution	DNS or network latency	Check DNS and network paths
SNI_* (various)	Internal network library synchronisation	Connection management	Usually benign

14.9 Batch mode and columnstore waits

Parallel **batch-mode** plans (columnstore and batch mode on rowstore) have their own synchronisation waits. They are normal parts of parallel execution; investigate only when they dominate a slow query and tuning (indexes, statistics) cannot reduce them. Microsoft's guidance for most of them: *if waiting is excessive and cannot be reduced by tuning the query, consider adjusting the cost threshold for parallelism or lowering the degree of parallelism.*

Wait type	What it means	Typical causes	What to do
BPSORT	Synchronising the sort of a dataset across threads in a batch-mode plan	Large batch sorts	Add indexes, reduce rows sorted, lower DOP
BMPALLOCATION	Synchronising allocation of a large bitmap filter	Large bitmap filters on big joins	Tune the query; lower DOP
BMPBUILD	Synchronising building of a large bitmap filter	Parallel hash join with bitmap	Same
BMPREPARTITION	Synchronising repartitioning of a large bitmap filter	Same	Same
BMPREPLICATION	Replicating a large bitmap filter across worker threads	Same	Same
HTBUILD	Synchronising building the hash table on the input side of a hash join or aggregation (SQL Server)	Large batch hash builds	Indexes/statistics; lower DOP
HTBUILD_AGG / HTBUILD_JOIN	Same, split into aggregation and join variants (Azure SQL, SQL Server 2025 / always-up-to-date)	Same	Same
HTDELETE (and _AGG / _JOIN)	Synchronising at the end of a hash join or aggregation	Same	Same
HTMEMO	Synchronising before scanning the hash table to output matches / non-matches	Hash join/aggregation	Same
HTREINIT	Synchronising before resetting a hash join/aggregation for the next partial join	Spilling batch hash joins	Fix spills (estimates, memory)

Wait type	What it means	Typical causes	What to do
HTREPARTITION	Synchronising repartitioning of the hash table	Large hash tables	Same
EXECUTION_PIPE_EVENT_INTERNAL	Synchronisation between producer and consumer parts of batch execution submitted through the connection context	Client submits batches via a pipe	Usually benign
COLUMNSTORE_BUILD_THROTTLE	Throttle for building columnstore index	Concurrent index builds	Internal
COLUMNSTORE_COLUMNDATASET_SESSION_LIST	Internal columnstore session list	Columnstore queries	Internal

14.10 Availability groups, mirroring and replication waits

HADR_SYNC_COMMIT

Category	Always On availability groups (synchronous commit)
What it means	A transaction on the primary is committing and is waiting for the **synchronous secondary replicas to harden the log**. The time covers sending the log, writing it on the secondary and the acknowledgement. This wait is expected on synchronous-commit groups; it also shows as the Transaction Delay counter.
Common causes	Network latency or bandwidth, slow log disk on the secondary, an overloaded secondary, many small commits, a replica far away (cross-region) in synchronous mode.
How to confirm	sys.dm_hadr_database_replica_states: log_send_queue_size, redo_queue_size, log_send_rate; Transaction Delay and Mirrored Write Transactions/sec counters; compare with WRITELOG on the primary.
How to fix / reduce	Improve secondary log disk latency and network, balance load, use asynchronous commit for distant replicas where data loss is acceptable, batch commits, remove unnecessary synchronous secondaries.
Concern level	Medium to **high** for write-heavy OLTP because it adds latency to each commit.

Wait type	What it means	Typical causes	What to do
HADR_DATABASE_FLOW_CONTROL	Waiting for messages to be sent to the partner when the maximum number of queued messages is reached; log scan runs faster than network sends	Slow network sends	Check bandwidth and latency to replica
HADR_TRANSPORT_FLOW_CONTROL	Waiting when the number of outstanding unacknowledged messages is over the flow control threshold (replica to replica)	Slow secondary or network	Investigate replica performance
HADR_LOGCAPTURE_WAIT	Waiting for log records to become available (new log generation or reading log not in cache)	Log scan caught up or reading from disk	Expected when caught up; check log read I/O if lag
HADR_GROUP_COMMIT	Transaction commit waiting to allow group commit of multiple log records into a single block	Normal optimisation	Expected
HADR_CLUSAPI_CALL	Thread switching to preemptive mode to call Windows failover clustering APIs	Cluster operations	Check WSFC health if high
HADR_LOGCAPTURE_SYNC, HADR_LOGPROGRESS_SYNC	Concurrency control around log capture and progress status	Partner state changes	Expected during state changes

Wait type	What it means	Typical causes	What to do
HADR_SYNCHRONIZING_THROTTLE	Commit processing allows a synchronising secondary to catch up before it transitions to synchronized	Replica catching up	Expected during catch-up
HADR_RECOVERY_WAIT_FOR_CONNECTION, HADR_RECOVERY_WAIT_FOR_UNDO	Secondary waiting to connect to primary before recovery; or waiting for undo to return to the common log point after failover	Failover or restart	Expected; track undo progress
HADR_DBSTATECHANGE_SYNC, HADR_REPLICAINFO_SYNC, HADR_PARTNER_SYNC	Concurrency control for replica/database state and the partner list	State changes	Internal
HADR_AG_MUTEX, HADR_AR_CRITICAL_SECTION_ENTRY, HADR_AR_MANAGER_MUTEX	AG DDL or cluster command waiting for exclusive access to AG/replica configuration or runtime state	Concurrent AG operations	Internal; avoid overlapping AG changes
HADR_BACKUP_BULK_LOCK, HADR_BACKUP_QUEUE	Backup on a secondary coordinating with the primary (BulkOp lock)	Backups on secondaries	Expected during backups
HADR_COMPRESSED_CACHE_SYNC	Cache of compressed log blocks used for several secondaries	Multiple replicas	Internal
HADR_DB_COMMAND, HADR_DB_OP_COMPLETION_SYNC, HADR_DB_OP_START_SYNC	Waiting for responses to conversational messages / serialised access to an availability database	AG operations	Internal
HADR_TRANSPORT_DBRLIST, HADR_TRANSPORT_SESSION	Transport layer replica list access; changing transport state	Transport	Internal
HADR_TDS_LISTENER_SYNC (_PROCESSING)	Listener start/stop requests and synchronisation	Listener changes	Internal
HADR_NOTIFICATION_DEQUEUE, HADR_NOTIFICATION_WORKER_*	Background tasks processing WSFC notifications	Cluster notifications	Ignore
HADR_WORK_QUEUE, HADR_WORK_POOL	AG background worker waiting for work / pool concurrency	Idle	Ignore (HADR_WORK_QUEUE is idle)
HADR_TIMER_TASK	Timer task lock and sleeps between periodic work	Periodic tasks	Ignore
HADR_DATABASE_VERSIONING_STATE, HADR_DATABASE_WAIT_FOR_TRANSITION_TO_VERSIONING	Readable secondary row-versioning state; a query waits for in-flight transactions to finish when the secondary became readable	Enabling readable secondary	Expected briefly
HADR_DATABASE_WAIT_FOR_RESTART	Waiting for the database to restart under AG control	Restart	Expected
HADR_FILESTREAM_*	FILESTREAM in AG: transport manager and I/O manager waits	FILESTREAM data in AG	Mostly idle/benign
HADR_XRF_STACK_ACCESS	Access to the extended recovery fork stack	Forks after failover	Internal
HADR_SEEDING_*, HADR_DBSEEDING*	Automatic seeding internals	Automatic seeding	Internal
PWAIT_HADR_*	Preemptive waits for AG operations (create, drop, failover, online/offline)	AG management commands	Expected during AG operations
PARALLEL_REDO_* and REDO_THREAD_PENDING_WORK	Parallel redo workers on secondaries waiting for work or coordinating	Redo on secondaries	Idle waits are benign; slow redo shows as redo_queue_size growing

Wait type	What it means	Typical causes	What to do
PREEMPTIVE_HADR_LEASE_MECHANISM	Lease manager scheduling for support diagnostics	Lease checks	Informational
DBMIRROR_EVENTS_QUEUE, DBMIRROR_WORKER_QUEUE, DBMIRROR_DBM_EVENT, DBMIRROR_DBM_MUTEX	Database mirroring idle/event queues	Mirroring	Ignore
DBMIRRORING_CMD	Task waiting for log records to be flushed to disk; expected to be held for long periods	Mirroring	Expected
REPL_*, SE_REPL_*, REPLICA_WRITES	Replication agents and snapshot/replica file writes	Replication	Check agent latency; mostly internal

14.11 Preemptive and external waits

PREEMPTIVE_OS_* (and other PREEMPTIVE_* waits)

Category	External / operating system calls
What it means	SQL Server's own scheduler is **cooperative**: threads yield voluntarily. When a thread calls code that SQL Server cannot schedule (a Windows API, a COM object, a driver, a network call) it switches to **preemptive mode**, where the OS scheduler controls it. The wait type name says *which kind of call*: for example PREEMPTIVE_OS_WRITEFILEGATHER (file growth), PREEMPTIVE_OS_AUTHENTICATIONOPS (Windows authentication).
Common causes	Depends on the suffix: slow storage or antivirus (file operations), slow domain controller or DNS (authentication and name resolution), certificate revocation checks (crypto), backup devices and VSS, CLR or COM components, cluster API calls, HTTP calls.
How to confirm	Identify the suffix, then check the matching external system: storage latency, domain controller response, DNS, antivirus exclusions, certificate infrastructure.
How to fix / reduce	Fix the external dependency: add antivirus exclusions, repair DC/DNS latency, enable instant file initialisation, pre-size files, fix cluster or backup-agent issues.
Concern level	Low individually; **high** when authentication or file-operation preemptive waits are long, because logins and growth events stall.

Preemptive family	What the calls do	Likely cause when slow
PREEMPTIVE_OS_AUTHENTICATIONOPS, AUTHORIZATIONOPS, LOOKUPACCOUNTSID, DSGETDCNAME, DOMAINSERVICESOPS, ACCEPTSECURITYCONTEXT, NETUSERGETGROUPS ...	Windows authentication, account lookups, group membership	Domain controller or network latency, trust problems
PREEMPTIVE_OS_CREATEFILE, WRITEFILE, WRITEFILEGATHER, FLUSHFILEBUFFERS, GETFILEATTRIBUTES, SETFILEPOINTER, DELETEFILE, MOVEFILE, COPYFILE, FILEOPS, DEVICEOPS ...	File system operations	Storage latency, antivirus, file growth, network shares
PREEMPTIVE_OS_CRYPTOPS, CRYPTACQUIRECONTEXT, VERIFYTRUST, VERIFYSIGNATURE, ...	Cryptography, certificate and signature checks	Certificate revocation list (CRL) lookups blocked or slow
PREEMPTIVE_COM_* and PREEMPTIVE_OLEDB*, PREEMPTIVE_ODBCOPS	COM objects, OLE DB / ODBC providers	Linked server providers, third-party components
PREEMPTIVE_CLUSAPI_CLUSTERRESOURCECONTROL, PREEMPTIVE_OS_CLUSTEROPS	Windows Server failover cluster API calls	Cluster health, resource control latency

Preemptive family	What the calls do	Likely cause when slow
PREEMPTIVE_DTC_*	Microsoft Distributed Transaction Coordinator operations	MSDTC service latency, distributed transactions
PREEMPTIVE_XE_* (CALLBACKEXECUTE, DISPATCHER, TARGETINIT, TIMERRUN ...)	Extended Events engine work	Event sessions with expensive targets; usually benign
PREEMPTIVE_VSS_*, PREEMPTIVE_CLOSEBACKUP*	Volume Shadow Copy and backup media operations	Snapshot backups, backup devices
PREEMPTIVE_HTTP_*, PREEMPTIVE_OS_WINSOCKOPS	HTTP and socket calls (for example backup to URL, external endpoints)	Network latency, endpoint availability
PREEMPTIVE_OS_SQLCLROPS, PREEMPTIVE_OS_LIBRARYOPS (LOADLIBRARY...)	CLR and library loading	First use of assemblies or extended procedures
PREEMPTIVE_SP_SERVER_DIAGNOSTICS, PREEMPTIVE_SERVER_STARTUP, PREEMPTIVE_SOSHOST ...	Internal health and hosting calls	Internal; usually benign

Microsoft documents most individual PREEMPTIVE_* waits as 'internal use only'. The practical rule: **read the suffix, name the external system, measure that system**.

14.12 Transactions, DTC and miscellaneous engine waits

Wait type	What it means	Typical causes	What to do
DTC, DTC_STATE, DTC_RESOLVE, DTC_ABORT_REQUEST, DTC_TMDOWN_REQUEST, DTC_WAITFOR_OUTCOME	Microsoft Distributed Transaction Coordinator state transitions and recovery: waiting for MSDTC availability, outcomes of prepared transactions, or ownership of a transaction	Distributed transactions, MSDTC service down or slow	Check MSDTC service and network; reduce distributed transactions
DTCNEW_*, DTCPNTSYNC	Internal MSDTC integration	Distributed transactions	Internal
MSQL_XACT_MGR_MUTEX, MSQL_XACT_MUTEX	Ownership of the session transaction manager / transaction usage mutex	MARS and multi-session transaction use	Rare; review MARS usage
TRANSACTION_MUTEX	Synchronising access to a transaction by multiple batches	MARS	Review application use of MARS
ENABLE_VERSIONING / DISABLE_VERSIONING	Waiting for transactions to finish when enabling or disabling snapshot isolation or RCSI	ALTER DATABASE options with open transactions	Expected; close long transactions or schedule
PVS_CLEANUP_LOCK	Persistent version store cleanup waiting for a lock (SQL Server 2019+ with accelerated database recovery)	Long active transactions blocking PVS cleanup	Find and end the oldest open transaction
DROPTEMP	Backs off between attempts to drop a temporary object after a failed attempt	Temp object cleanup retries	Check tempdb health
GHOSTCLEANUPSYNCMGR, GHOSTCLEANUP_UPDATE_STATS	Internal ghost record cleanup coordination	Heavy deletes	Internal
DUMP_LOG_COORDINATOR	Main task waiting for a subtask to generate data; ordinarily does not occur	Unexpected blockage	Investigate the subtask
CLEAR_DB	Operations that change database state (open/close)	Database open/close	Expected
BAD_PAGE_PROCESS	Suspect page logger throttling itself to run at most every 5 seconds	Excess suspect pages	**Investigate corruption** (msdb.dbo.suspect_pages, DBCC CHECKDB)
ERROR_REPORTING_MANAGER	Synchronising concurrent error log initialisations	Startup	Benign
ASSEMBLY_LOAD, DLL_LOADING_MUTEX	Loading assemblies / XML parser DLL	First use	Benign
AUDIT_GROUPCACHE_LOCK, AUDIT_LOGINCACHE_LOCK, AUDIT_ON_DEMAND_TARGET_LOCK, AUDIT_XE_SESSION_MGR	SQL Server Audit internal caches and Extended Events session management	Auditing configuration changes	Usually benign; heavy auditing can add overhead
BUILTIN_HASHKEY_MUTEX	Initialising internal data structures after startup	Startup	Benign
EXTERNAL_RG_UPDATE, EXTERNAL_SCRIPT_*	External resource governance and R/Python external script service	External scripts	Check ML Services workload

14.13 Service Broker, CLR, full-text, FILESTREAM, in-memory OLTP and Query Store

Wait type	What it means	Typical causes	What to do
BROKER_RECEIVE_WAITFOR	WAITFOR RECEIVE waiting: no messages in the queue, or lock contention preventing receipt	Idle Service Broker readers	Usually benign; counts as 'User Wait'
BROKER_TRANSMITTER, BROKER_TO_FLUSH, BROKER_EVENTHANDLER, BROKER_TASK_STOP, BROKER_INIT, BROKER_MASTERSTART, BROKER_SHUTDOWN, BROKER_REGISTERALLENDPOINTS	Service Broker background tasks waiting for work, flushing, starting or stopping (the transmitter shows 2 waits even if Broker is unused)	Background	Ignore
BROKER_CONNECTION_RECEIVE_TASK, BROKER_ENDPOINT_STATE_MUTEX, BROKER_SERVICE	Serialised access to endpoints or service destination lists	Many dialogs	Rare
CLR_AUTO_EVENT, CLR_MANUAL_EVENT	CLR task waiting for an event; long waits are typical and not a problem	CLR host idle	Ignore
CLR_CRST, CLR_MONITOR, CLR_RWLOCK_READER/WRITER, CLR_SEMAPHORE, CLR_JOIN, CLR_TASK_START	CLR code waiting on critical sections, monitors, locks, semaphores or task completion	Contention inside CLR assemblies	Review the CLR code
CLRHOST_STATE_ACCESS, CLR_MEMORY_SPY	CLR host setup/teardown and virtual memory tracking	CLR initialisation	Benign
MSSEARCH, FT_RESTART_CRAWL, FULLTEXT GATHERER, FT_* (FT_IFTS_*, FT_MASTER_MERGE, FT_IFTSHC_MUTEX ...)	Full-text search and crawl operations; MSSEARCH measures the duration of full-text calls, not contention	Full-text indexing and queries	Tune full-text population; most FT_* waits are internal
FT_IFTS_SCHEDULER_IDLE_WAIT	Full-text scheduler idle	Background	Ignore
FS_FC_RWLOCK, FS_GARBAGE_COLLECTOR_SHUTDOWN, FS_HEADER_RWLOCK, FS_LOGTRUNC_RWLOCK, FSA_FORCE_OWN_XACT, FSAGENT, FSTR_CONFIG_*	FILESTREAM garbage collection, headers, log truncation and agent resources	FILESTREAM workloads	Check FILESTREAM I/O and transactions
FFT_* (FileTable)	FileTable internal structures	FileTable use	Internal
WAIT_XTP_* and XTP_* (for example WAIT_XTP_HOST_WAIT, WAIT_XTP_RECOVERY, WAIT_XTP_OFFLINE_CKPT_*)	In-Memory OLTP engine checkpoint, recovery and hosting waits	In-memory tables	Mostly internal; slow recovery shows as long database recovery
QDS_ASYNC_QUEUE, QDS_PERSIST_TASK_MAIN_LOOP_SLEEP, QDS_CLEANUP_STALE_QUERIES_TASK_MAIN_LOOP_SLEEP, QDS_SHUTDOWN_QUEUE, QDS_STMT	Query Store background tasks (persisting data, cleaning up stale queries) idle or flushing	Query Store active	Ignore; very heavy Query Store workloads can show contention on QDS_* in rare cases

Wait type	What it means	Typical causes	What to do
XE_DISPATCHER_WAIT, XE_DISPATCHER_JOIN, XE_TIMER_EVENT, XE_BUFFERMGR_*, XE_LIVE_TARGET_TVF, XE_SERVICES_MUTEX	Extended Events dispatcher, timer, buffer and live-target tasks	XEvent sessions	Ignore (idle/background)
SQLTRACE_BUFFER_FLUSH, SQLTRACE_INCREMENTAL_FLUSH_SLEEP, SQLTRACE_WAIT_ENTRIES, SQLTRACE_LOCK, TRACEWRITE	Server-side trace buffer flushing and rowset trace consumers	Traces running	Ignore the flush waits; TRACEWRITE means a slow trace consumer
PERFORMANCE_COUNTERS_RWLOCK	Performance counter structure lock	Counter collection	Internal
KSOURCE_WAKEUP	Service control task waiting for requests from the Service Control Manager	Idle	Ignore
SERVER_IDLE_CHECK, SLEEP_BPOOL_FLUSH, SLEEP_*	Idle/sleep waits of system tasks (database startup, buffer pool flush, etc.)	Background	Ignore

14.14 Background, idle and benign waits

These waits record time that **system tasks spend idle or sleeping**. They often show the largest totals in sys.dm_os_wait_stats and mean nothing about performance. Exclude them before ranking waits.

Wait type(s)	What the idle task is waiting for
SLEEP_TASK, SLEEP_SYSTEMTASK, SLEEP_BPOOL_FLUSH, SLEEP_DBSTARTUP, SLEEP_DCOMSTARTUP, SLEEP_MASTERDBREADY, SLEEP_MASTERMDREADY, SLEEP_MASTERUPGRADED, SLEEP_MSDBSTARTUP, SLEEP_TEMPDBSTARTUP	System tasks sleeping, or waiting for system databases to be ready during startup
LAZYWRITER_SLEEP, CHECKPOINT_QUEUE, CHKPT	Lazy writer sleeping; checkpoint thread waiting for the next request or startup signal
LOGMGR_QUEUE, ONDEMAND_TASK_QUEUE, DISPATCHER_QUEUE_SEMAPHORE, SOS_WORK_DISPATCHER	Log writer, background and dispatcher workers waiting for requests
REQUEST_FOR_DEADLOCK_SEARCH	Deadlock monitor waiting for its next search interval
XE_TIMER_EVENT, XE_DISPATCHER_WAIT, XE_DISPATCHER_JOIN, XE_LIVE_TARGET_TVF	Extended Events background threads
SQLTRACE_BUFFER_FLUSH, SQLTRACE_INCREMENTAL_FLUSH_SLEEP, SQLTRACE_WAIT_ENTRIES	Trace subsystem flushing and waiting
BROKER_TO_FLUSH, BROKER_TASK_STOP, BROKER_EVENTHANDLER, BROKER_TRANSMITTER, BROKER_RECEIVE_WAITFOR	Service Broker background tasks and idle receivers
FT_IFTS_SCHEDULER_IDLE_WAIT, FSAGENT	Full-text scheduler idle; FILESTREAM agent resource
DIRTY_PAGE_POLL	Internal dirty page polling

Wait type(s)	What the idle task is waiting for
HADR_FILESTREAM_IOMGR_IOCOMPLETION, HADR_WORK_QUEUE, HADR_TIMER_TASK, HADR_NOTIFICATION_DEQUEUE, HADR_LOGCAPTURE_WAIT	Availability group background workers idle or between work items
PARALLEL_REDO_WORKER_WAIT_WORK, PARALLEL_REDO_DRAIN_WORKER, PARALLEL_REDO_LOG_CACHE, PARALLEL_REDO_TRAN_LIST, PARALLEL_REDO_WORKER_SYNC, REDO_THREAD_PENDING_WORK	Parallel redo workers on secondaries waiting for log to apply
DBMIRROR_EVENTS_QUEUE, DBMIRROR_WORKER_QUEUE, DBMIRROR_DBM_EVENT, DBMIRRORING_CMD	Database mirroring workers idle or waiting to flush
QDS_ASYNC_QUEUE, QDS_PERSIST_TASK_MAIN_LOOP_SLEEP, QDS_CLEANUP_STALE_QUERIES_TASK_MAIN_LOOP_SLEEP, QDS_SHUTDOWN_QUEUE	Query Store background tasks
SP_SERVER_DIAGNOSTICS_SLEEP, PREEMPTIVE_SP_SERVER_DIAGNOSTICS, PWAIT_ALL_COMPONENTS_INITIALIZED	Health diagnostics collection and startup initialisation
WAITFOR, WAIT_FOR_RESULTS, WAITFOR_TASKSHUTDOWN	Sessions that run WAITFOR on purpose; waiting for results of an async operation
CLR_AUTO_EVENT, CLR_MANUAL_EVENT, CLR_SEMAPHORE	CLR host tasks waiting for events (long waits are typical)
KSOURCE_WAKEUP, SERVER_IDLE_CHECK, VDI_CLIENT_OTHER	Service control signal; instance idle check; backup client (VDI) idle
PREEMPTIVE_HADR_LEASE_MECHANISM, PREEMPTIVE_XE_*	Preemptive housekeeping calls for lease checks and Extended Events

Conditionally benign: some waits are harmless in normal amounts but matter when they dominate or are very long: `CXCONSUMER`, `ASYNC_IO_COMPLETION` and `BACKUPTHREAD` during backups, `HADR_CLUSAPI_CALL`, `MEMORY_ALLOCATION_EXT`, and `PREEMPTIVE_OS_*`. Always judge them against a baseline.

14.15 Internal-use and undocumented waits

Microsoft marks many wait types as *Internal use only* or *Identified for informational purposes only; not supported; future compatibility is not guaranteed*. They exist for the engine's own synchronisation and for Microsoft support. Do not chase them unless they dominate a real slowdown.

Family (prefix)	Area	How to treat
FABRIC_*, HADR_FABRIC_*	Availability group / fabric integration	Internal; relevant only to support
FFT_*	FileTable / filter driver	Internal
FT_IFTS_* (SQL Server 2022 CU1+), FT_*	Full-text indexing and search internals	Internal; tune population and queries instead
FILESTREAM_*, FS_*	FILESTREAM engine	Internal; check FILESTREAM I/O
GLOBAL_QUERY_*, GLOBAL_TRAN_*	Distributed/global query and transaction internals	Internal
HADR_SEEDING_*, HADR_DBSEEDING*	Automatic seeding	Internal

Family (prefix)	Area	How to treat
LOGPOOL_*, LOGCAPTURE_*, DIRECTLOGCONSUMER_*	Log pool and log consumers for AG, replication, CDC	Internal
PWAIT_*	Preemptive/background coordination waits (many for AG operations)	Internal
PARALLEL_REDO_*	Parallel redo on secondaries	Idle ones are benign; slow redo is seen through redo queue
BROKER_*	Service Broker internals	Internal or idle
DTCNEW_*, KTM_*	Distributed transaction / kernel transaction manager	Internal
EXTERNAL_SCRIPT_*, EXTERNAL_RG_UPDATE	External scripts and resource governance	Internal
WAIT_XTP_*, XTP*	In-Memory OLTP	Internal
ABR, EC, GUARDIAN, INTERNAL_TESTING, FAILPOINT, PREEMPTIVE_TESTING and similar	Test or reserved types	Ignore

What to do with a wait you do not recognise

- **Search the Microsoft documentation** for sys.dm_os_wait_stats; many waits have a one-line description even if marked internal.
- **Read the prefix**: LCK_ = lock, LATCH/PAGELATCH/PAGEIOLATCH = latch, HADR_ = availability groups, PREEMPTIVE_ = outside the engine, SLEEP_/QUEUE/_IDLE = idle, CX = parallelism, HT/BMP/BPSORT = batch mode.
- **Check how it behaves**: idle waits have long average times and no active sessions; real waits correlate with slow requests (sys.dm_exec_requests).
- **Correlate** with the resource it points to (disk, CPU, memory, log, network) using DMVs and perfmon counters.
- **Capture with Extended Events** (sqlos.wait_info) filtered on the wait type to see which queries and how long.
- If it is long, repeatable and unexplained, **open a support case** with the wait statistics, version and build number.

```
-- Map numeric wait IDs from Extended Events to names, and find all HADR waits (example)
SELECT map_key, map_value FROM sys.dm_xe_map_values WHERE name = 'wait_types' AND map_value LIKE
    '%HADR%' ORDER BY map_key;
-- How many wait types does this instance expose?
SELECT COUNT(*) AS wait_types FROM sys.dm_os_wait_stats;
```

14.16 Azure SQL and Managed Instance notes

- **Log rate governance:** LOG_RATE_GOVERNOR, POOL_LOG_RATE_GOVERNOR, INSTANCE_LOG_RATE_GOVERNOR and HADR_THROTTLE_LOG_RATE_* indicate the service tier's log rate limit, or a replica that cannot keep up. Scale the tier or reduce log volume.
- **Per-database waits:** use sys.dm_db_wait_stats in Azure SQL Database for waits scoped to the database.
- **Batch mode waits:** HTBUILD_AGG/JOIN and HTDELETE_AGG/JOIN appear in Azure SQL and in SQL Server 2025 builds.
- **Permissions:** reading wait DMVs needs VIEW DATABASE STATE or the ##MS_ServerStateReader## role on most service objectives.

15. Wait-Based Troubleshooting Playbooks

Waits tell you *which resource* hurts. These playbooks turn a dominant wait into a sequence of checks and fixes. Always confirm with a second source (DMV, counter or plan) before changing anything.

15.1 First step: group waits by resource

```
WITH w AS (
  SELECT wait_type, wait_time_ms, signal_wait_time_ms FROM sys.dm_os_wait_stats
  WHERE wait_type NOT IN
      ('SLEEP_TASK','LAZYWRITER_SLEEP','CHECKPOINT_QUEUE','XE_TIMER_EVENT','XE_DISPATCHER_WAIT','REQUEST
      _FOR_DEADLOCK_SEARCH',
        'SQLTRACE_INCREMENTAL_FLUSH_SLEEP','BROKER_TO_FLUSH','BROKER_EVENTHANDLER','DIRTY_PAGE_POLL','LO
            GMGR_QUEUE','ONDEMAND_TASK_QUEUE',
        'WAITFOR','SP_SERVER_DIAGNOSTICS_SLEEP','QDS_PERSIST_TASK_MAIN_LOOP_SLEEP','QDS_ASYNC_QUEUE','FT
            _IFTS_SCHEDULER_IDLE_WAIT',
        'SLEEP_SYSTEMTASK','BROKER_TASK_STOP','BROKER_TRANSMITTER','SQLTRACE_BUFFER_FLUSH','CLR_AUTO_EVE
            NT','CLR_MANUAL_EVENT',
        'DISPATCHER_QUEUE_SEMAPHORE','HADR_WORK_QUEUE','HADR_TIMER_TASK','HADR_NOTIFICATION_DEQUEUE','HA
            DR_LOGCAPTURE_WAIT','REDO_THREAD_PENDING_WORK')
    AND wait_time_ms > 0)
SELECT category = CASE
    WHEN wait_type = 'SOS_SCHEDULER_YIELD' THEN 'CPU'
    WHEN wait_type = 'THREADPOOL' THEN 'Worker threads'
    WHEN wait_type LIKE 'CX%' OR wait_type IN ('EXCHANGE','EXECSYNC') THEN 'Parallelism'
    WHEN wait_type LIKE 'RESOURCE_SEMAPHORE%' OR wait_type LIKE 'CMEM%' THEN 'Memory'
    WHEN wait_type LIKE 'PAGEIOLATCH%' OR wait_type IN
        ('IO_COMPLETION','ASYNC_IO_COMPLETION','WRITE_COMPLETION') THEN 'Data I/O'
    WHEN wait_type IN ('WRITELOG','LOGBUFFER','LOGMGR_RESERVE_APPEND') OR wait_type LIKE
        '%LOG_RATE_GOVERNOR' THEN 'Log'
    WHEN wait_type LIKE 'LCK_M_%' THEN 'Locking'
    WHEN wait_type LIKE 'PAGELATCH%' OR wait_type LIKE 'LATCH_%' THEN 'Latches'
    WHEN wait_type IN ('ASYNC_NETWORK_IO','NET_WAITFOR_PACKET','OLEDB') THEN 'Network / client'
    WHEN wait_type LIKE 'HADR_%' OR wait_type LIKE 'DBMIRROR%' THEN 'HA / replication'
    WHEN wait_type LIKE 'PREEMPTIVE_%' THEN 'Preemptive / external'
    ELSE 'Other' END,
  SUM(wait_time_ms) / 1000.0 AS wait_s, SUM(signal_wait_time_ms) / 1000.0 AS signal_s
FROM w GROUP BY CASE
    WHEN wait_type = 'SOS_SCHEDULER_YIELD' THEN 'CPU' WHEN wait_type = 'THREADPOOL' THEN 'Worker
        threads'
    WHEN wait_type LIKE 'CX%' OR wait_type IN ('EXCHANGE','EXECSYNC') THEN 'Parallelism'
    WHEN wait_type LIKE 'RESOURCE_SEMAPHORE%' OR wait_type LIKE 'CMEM%' THEN 'Memory'
    WHEN wait_type LIKE 'PAGEIOLATCH%' OR wait_type IN
        ('IO_COMPLETION','ASYNC_IO_COMPLETION','WRITE_COMPLETION') THEN 'Data I/O'
    WHEN wait_type IN ('WRITELOG','LOGBUFFER','LOGMGR_RESERVE_APPEND') OR wait_type LIKE
        '%LOG_RATE_GOVERNOR' THEN 'Log'
    WHEN wait_type LIKE 'LCK_M_%' THEN 'Locking' WHEN wait_type LIKE 'PAGELATCH%' OR wait_type LIKE
        'LATCH_%' THEN 'Latches'
    WHEN wait_type IN ('ASYNC_NETWORK_IO','NET_WAITFOR_PACKET','OLEDB') THEN 'Network / client'
    WHEN wait_type LIKE 'HADR_%' OR wait_type LIKE 'DBMIRROR%' THEN 'HA / replication'
    WHEN wait_type LIKE 'PREEMPTIVE_%' THEN 'Preemptive / external' ELSE 'Other' END
ORDER BY wait_s DESC;
```

15.2 CPU pressure

Signs	Checks	Actions
SOS_SCHEDULER_YIELD high; signal waits above 20-25% of total; Windows CPU near 100%; runnable tasks > 0	sys.dm_os_schedulers (runnable_tasks_count); top CPU queries; compile/recompile rates; hypervisor ready time	Tune top CPU queries; fix scans and implicit conversions; reduce parallelism oversubscription; check power plan; add or resize CPUs

```
SELECT scheduler_id, cpu_id, current_tasks_count, runnable_tasks_count, work_queue_count,
    active_workers_count
FROM sys.dm_os_schedulers WHERE scheduler_id < 255 ORDER BY runnable_tasks_count DESC;
```

15.3 Data I/O pressure

Signs	Checks	Actions
PAGEIOLATCH_SH high; read latency above 10-20 ms; low page life expectancy	sys.dm_io_virtual_file_stats; Perfmon Avg. Disk sec/Read; queries with highest physical reads	Reduce reads (indexes, plan fixes); add memory; faster storage; spread files; check cache hit ratio and read-ahead behaviour

15.4 Transaction log bottleneck

Signs	Checks	Actions
WRITELOG, LOGBUFFER high; log write latency above 2-5 ms	Log file write latency; Log Flush Wait Time; commits/sec; VLF count; HADR_SYNC_COMMIT	Faster log storage; larger batches per transaction; reduce logging; delayed durability only when acceptable; resolve AG commit latency
LOGMGR_RESERVE_APPEND, error 9002	log_reuse_wait_desc; open transactions; log backups; disk space	Fix the reuse blocker; add log space; back up log; end long transactions

15.5 Blocking and locking

Signs	Checks	Actions
LCK_M_* waits with long durations; blocked sessions; timeouts	Head blocker and its SQL; open transaction age; isolation level; lock escalation; deadlock graph	RCSI/SNAPSHOT; indexes for UPDATE/DELETE; shorter transactions; batching; consistent access order; low-priority DDL

15.6 Memory: grants and compilation

Signs	Checks	Actions
RESOURCE_SEMAPHORE; queries not starting	sys.dm_exec_query_memory_grants; spills; estimates; concurrency of reports	Fix estimates; grant hints; feedback; Resource Governor; add memory
RESOURCE_SEMAPHORE_QUERY_COMPILE, CMEMTHREAD	Compilations per second; query_hash variants; plan cache composition	Parameterise; optimize for ad hoc workloads; avoid cache clears

15.7 tempdb allocation contention

Signs	Checks	Actions
PAGELATCH_UP/EX/SH on tempdb pages 2:1:1, 2:1:2, 2:1:3 and PFS intervals	sys.dm_os_waiting_tasks resource_description; number and size of tempdb files; temp table churn	Equal-sized tempdb files (commonly up to 8 or the core count); cache temp objects; memory-optimized tempdb metadata (2019+); avoid unnecessary temp objects

15.8 Network and client

Signs	Checks	Actions
ASYNC_NETWORK_IO dominates; queries show 'suspended' while returning rows	Rows returned; client CPU; application read loop; network path	Return fewer rows; fast readers; fix client resources; check drivers and network

15.9 Availability group latency

Signs	Checks	Actions
HADR_SYNC_COMMIT, WRITELOG; growing log send or redo queue	sys.dm_hadr_database_replica_states (log_send_queue_size, redo_queue_size); network latency; secondary storage and CPU	Faster secondary log disk and network; reduce synchronous replicas; async for distant replicas; fix redo bottlenecks

15.10 Connecting plans to waits

Every wait is the *result* of something in the plan or the environment. This table links what you see in a plan to the waits it tends to produce, so you can move from either side to the other.

In the plan	Waits you will often see	Why
Large scans, Key Lookups with many executions	PAGEIOLATCH_SH, ASYNC_NETWORK_IO (if results large)	Many pages read from disk; client reading rows
Hash Match or Sort spill (Hash/Sort Warning)	IO_COMPLETION, PAGEIOLATCH on tempdb, PAGELATCH on tempdb	Spill writes and reads in tempdb
Huge memory grant or queued grant	RESOURCE_SEMAPHORE	Waiting to reserve working memory
Parallel plan with skew or serial zone	CXPACKET, CXCONSUMER, CXSYNC_PORT, EXECSYNC	Threads wait for each other at exchanges
Many parallel queries at once	THREADPOOL, SOS_SCHEDULER_YIELD	Worker thread and CPU shortage
Batch mode hash join/aggregate or sort	HTBUILD, HTDELETE, BPSORT, BMP*	Batch-mode synchronisation
UPDATE/DELETE with wide plan, many indexes, triggers	LCK_M_X, WRITELOG, PAGELATCH_EX	Locks, log flushes and page contention
INSERT at the end of an identity/datetime clustered key	PAGELATCH_EX (last page), WRITELOG	Hot page and log flush
Remote Query / linked server operators	OLEDB, MSQL_DQ, ASYNC_NETWORK_IO	Waiting on the remote provider
Plan compile of complex ad hoc statements	RESOURCE_SEMAPHORE_QUERY_COMPILE, CMEMTHREAD	Compilation gateways and memory object contention
Serializable range scans; HOLDLOCK	LCK_M_RS_*, LCK_M_RIn_*, LCK_M_RX_*	Range locks
DDL and index rebuilds on busy tables	LCK_M_SCH_M, LCK_M_SCH_S	Schema modification vs stability locks
Scalar UDF row by row	SOS_SCHEDULER_YIELD (CPU), sometimes PREEMPTIVE_* for CLR	Per-row CPU and serial execution
Big result sets returned to a client	ASYNC_NETWORK_IO	Client consumption speed

```
-- Waits per query and plan from Query Store (shows which wait categories hurt each plan)
SELECT TOP 20 q.query_id, p.plan_id, ws.wait_category_desc, SUM(ws.total_query_wait_time_ms) AS
    total_wait_ms,
       SUM(ws.total_query_wait_time_ms) * 1.0 / NULLIF(SUM(rs.count_executions), 0) AS wait_ms_per_exec
FROM sys.query_store_wait_stats ws
JOIN sys.query_store_plan p ON p.plan_id = ws.plan_id
JOIN sys.query_store_query q ON q.query_id = p.query_id
LEFT JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id AND rs.runtime_stats_interval_id =
    ws.runtime_stats_interval_id
GROUP BY q.query_id, p.plan_id, ws.wait_category_desc ORDER BY total_wait_ms DESC;
```

Appendix A: Operator Quick Reference

Operator	Good sign	Warning sign
Index Seek	Selective seek predicate on leading key	Huge row counts or residual Predicate removing most rows
Index / Clustered Index Scan	Small table, or most rows needed	Rows Read far larger than Actual Rows
Key / RID Lookup	Few executions	Hundreds of thousands of executions
Nested Loops	Small outer, indexed inner	Large outer, inner scan, estimate of 1 execution
Merge Join	Presorted inputs, one-to-many	Sorts below it, many-to-many
Hash Match	Large unsorted inputs, in memory	Hash Warning, bad build side, huge grant
Adaptive Join	Actual type matches data size	Large grant for a nested-loop outcome
Stream Aggregate	Ordered input	Sort feeding it for many rows
Hash Match (Aggregate)	Moderate group count	Spill from underestimated groups
Sort / TopN Sort	Small, or index-ordered	Spill, huge row counts, wide rows
Compute Scalar	Cheap expressions	Hidden scalar UDF
Filter	Early, cheap	Late filter after expensive joins
Table / Index Spool	Rewinds reuse data	Rebinds rebuild per row; missing index
Eager Spool (read plan)	-	Almost always a sign of a missing index or rewrite need
Parallelism (Gather / Repartition)	Even thread distribution	Skew, order-preserving, spills
Table-valued function	Inline TVF expanded (no operator)	Multi-statement TVF with fixed estimate
Remote Query / Scan	Filtered remotely	Large remote result sets
Update/Insert/Delete operators	Narrow plan, few indexes	Wide plan, many indexes, triggers, large batches
Assert	Constraint checks	Scalar subquery returning many rows
Window Spool	ROWS frame in memory	RANGE default on disk

Appendix B: Diagnostic Toolkit

Object	Purpose
sys.dm_exec_query_stats, sys.dm_exec_procedure_stats	Aggregated runtime stats of cached statements and procedures
sys.dm_exec_query_plan, sys.dm_exec_text_query_plan, sys.dm_exec_query_plan_stats	Cached plan, per-statement plan, last actual plan
sys.dm_exec_query_statistics_xml, sys.dm_exec_query_profiles	Live plan and operator progress
sys.dm_exec_requests, sys.dm_exec_sessions, sys.dm_exec_connections	Current activity, waits, blocking, connection details
sys.dm_os_wait_stats, sys.dm_exec_session_wait_stats, sys.dm_os_waiting_tasks	Wait statistics at instance, session and task level

Object	Purpose
sys.dm_os_latch_stats, sys.dm_os_spinlock_stats, sys.dm_os_schedulers	Latch classes, spinlocks, CPU queues
sys.dm_io_virtual_file_stats, sys.dm_db_file_space_usage	File I/O latency and tempdb space
sys.dm_exec_query_memory_grants, sys.dm_os_memory_clerks	Memory grants and memory usage
sys.dm_tran_locks, sys.dm_tran_active_transactions	Locks and open transactions
sys.dm_db_missing_index_*, sys.dm_db_index_usage_stats, sys.dm_db_index_operational_stats	Index suggestions and usage
Query Store views (sys.query_store_*)	Query, plan, runtime stats, waits, hints, feedback
sys.dm_hadr_database_replica_states	AG send/redo queues and lag
Extended Events: sqlos.wait_info, query_post_execution_showplan, xml_deadlock_report, system_health	Targeted capture of waits, plans and deadlocks
SET STATISTICS IO / TIME / XML	Per-statement reads, timing and plan

Appendix C: Glossary

Term	Meaning
Cardinality	The number of rows an operator produces or consumes
Cardinality estimator (CE)	Optimizer component that predicts row counts from statistics
Cost	Unit-less estimate used by the optimizer to compare plans
Iterator	An operator implemented as Open/GetRow/Close
Row mode / Batch mode	Processing one row at a time / about 900 rows at a time
SARGable	A predicate that can use an index seek
Seek / Scan	Navigate an index to specific keys / read the whole index or range
Residual predicate	A filter applied to rows already read rather than used to seek
Spill	An operator writing to tempdb because its memory grant was too small
Memory grant	Working memory reserved for a query before it runs
Parameter sniffing	Compiling a plan using the first parameter values seen
Worker thread	A SQL Server thread running a task
Scheduler / quantum	CPU scheduling unit / the time slice (about 4 ms) a thread runs before yielding
Signal wait	Time a thread spends in the runnable queue waiting for a CPU
Latch	Lightweight internal lock on in-memory structures
Lock	Transactional lock protecting data consistency
Spinlock	Very light synchronisation that spins on a CPU instead of waiting
Preemptive mode	A thread calling code outside SQL Server's cooperative scheduler
Query Store	Built-in history of queries, plans, runtime stats and waits

Term	Meaning
IQP	Intelligent Query Processing: automatic plan improvements by version

Appendix D: Where to Learn More

Operators, wait types and IQP features change with each SQL Server release. Use this guide for understanding and method, and confirm details for your build in Microsoft's documentation.

Topic	Where to look (learn.microsoft.com and community sources)
Operators	'Showplan logical and physical operators reference'
Wait types	'sys.dm_os_wait_stats (Transact-SQL)' and the SQLskills / Microsoft CSS community wait type libraries
Joins	'Joins' in the Query processing architecture guide
Query Store	'Monitoring performance by using the Query Store' and Query Store hints
IQP	'Intelligent query processing in SQL databases' and the per-feature articles
Statistics and cardinality	'Statistics' and 'Cardinality Estimation (SQL Server)'
Latch and spinlock contention	'Diagnose and resolve latch contention' and 'Diagnose and resolve spinlock contention'
Memory grants	'Troubleshoot slow queries that result from memory grant waits'